République Tunisienne

Ministère de l'enseignement Supérieur

*Et de la Recherche Scientifiq*ue

Institut National des Sciences Appliquées et de Technologie

Support de Cours

Module: Microcontrôleur et DSP

3ème Année IMI

Enseignant:

Slim Yacoub

Table des Matières

Chapitre 1: Les Processeurs.

1. Introduction. 3
2. Les processeurs.
3. Principe de fonctionnement d'un processeur. 4
   - 3.1 Introduction.
   - 3.2 Fonctionnement d'un processeur. 5
   - 3.3 Exemple de fonctionnement d'un processeur. 6
   - 3.4 Les Bus. 7
4. Structure Von Neumann et Harvard.
   - 4.1. Structure Von Neumann.
   - 4.2 Structure Harvard.
5. Différents modes de fonctionnement des processeurs. 9
   - 5.1. Principe de fonctionnement du Pipeline.
   - 5.2. Les aléas de fonctionnement d'un pipeline. 11
     - 5.2.1 Les Aléas de Donnée.
     - 5.2.2 Les Aléas structurels. 12
     - 5.2.3 Les Aléas de contrôle.
   - 5.3 Mode super scalaire.
6. Format de calcul des DSP. 13
   - 6.1 Codage en virgule fixe.
   - 6.2 Codage en virgule flottante. 14

Chapitre 2: Les Processeurs Numériques de signaux

1. Introduction. 15
2. Filtrage numérique avec un DSP. 16
   - 2.1. Chaîne typique d un système de traitement numérique du signal.
   - 2.2. Architecture générale d'un DSP.
3. Fonctionnement d'un DSP. 17
   - 3.1 La mémoire Cache. 18
   - 3.2. L'unité arithmétique et logique. 19
     - 3.2.1 Les registres de l'ALU.
     - 3.2.2 Fonctions standard de l'ALU. 20
   - 3.3 Les différents modes d'adressage. 21
     - 3.3.1 Adressage standard. 22
     - 3.3.2 Adressage circulaire ou modulo. 23
   - 3.4 La configuration MAC *Multiply and ACcumulate*. 24
   - 3.5. Le contrôleur d'interruption. 25
   - 3.6 Les mémoires. 27
     - 3.6.1 Accès direct a la mémoire. 28

3.7 Port série. 29
3.7.1 Exemple de configuration des ports série de l'ADSP 21xx.
4. Langage assembleur de la famille ADSP21xx. 31
4.1 La famille ADSP2100.
4.1.1Exemple d'une instruction type ALU/MAC. 32
5. Implantation d'algorithmes de traitement numérique du signal sur DSP
5.1 Filtre FIR. 33
5.1.1 Implantation du filtre FIR. 34
5.2 Systèmes a réponse impulsionnelle infinie (RII ou IIR). 37
5.2.1 Implémentation du filtre IIR. 38
5.3 L'algorithme LMS. 39
5.3.1 Structure de L'algorithme LMS.
5.3.2. Implantation de L algorithme LMS. 41

Chapitre 3: Les microcontrôleurs

1. Introduction. 44
2. Les microcontrôleurs de MOTOROLA.

3 Description du microcontrôleur 68332. 45
3.1 Constituant du microcontrôleur 68332. 46
3.1.1 Le module processeur CPU32.
3.1.1.1 Les Instructions du CPU. 47
3.1.2 Le module d'intégration SIM . 50
3.1.3 Le module interface série (QSM) 51
3.1.4 Le module mémoire RAM. 52
3.1.5 Le module TPU.
3.1.5.1 Adressage du registre TPU. 53
3.1.5.2 Les fonctions de gestion de temps du module TPU.
3.1.5.3 Exemple de configuration des canaux en mode entrée/ sortie TOR. 55
3.1.5.4 Quelques Registres de configuration du module TPU. 56

References bibliographiques. 60

Chapitre 1: Les Processeurs

1 Introduction.

Les premiers microprocesseurs ont été fabriqués au début des années 1970 par la société intel. Depuis ils n'ont cessé d'évoluer en terme de puissance de calcul, de fonctions, et de miniaturisation. Pour pouvoir fonctionner dans une application un micro processeur a besoin de certains éléments telque une mémoire ROM (Read Only Memory) a lecture seulement qui renferme le programme à être exécuter, une mémoire vive (RAM : Random Access Memory) qui permet de sauvegarder temporairement des données, des bus a l'aide des quels les différents éléments communiquent, une horloge et des périphériques d'entrée sortie permettant au microprocesseur de communiquer avec l'extérieur.

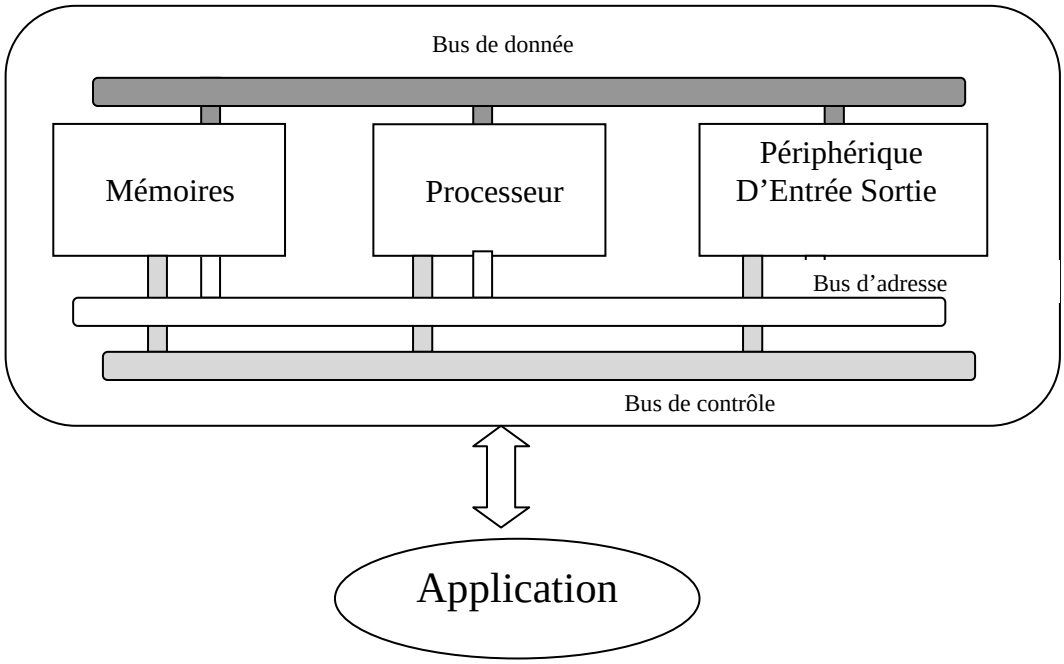


Figure1 Le microprocesseur et ces périphériques.

2 Les processeurs.

Les processeurs sont classés en deux grandes catégories les généralistes telque les microcontrôleurs et les microprocesseurs et les spécifiques telque les DSP, les ASICs et les FPGA.

Un microcontrôleur est un circuit intégré comprenant essentiellement un microprocesseur ses mémoires et des éléments personnalisés (ADC, DAC,…) selon l'application. Ils sont souvent utilisés dans l'élaboration des systèmes embarqués nécessitant des traitements spécialisés tel que les lecteurs MP3 les GPS et les autoradios. Ils disposent en général d'une architecture CISC (Complex Instruction Set Computer) caractérisée par un grand choix d'instructions complexe.

Alors que les DSP servent surtout pour les applications de traitement numérique du signal ce sont des microprocesseurs munis d' un ensemble de fonctions dédiées au traitement numérique du signal, ils possèdent en général une architecture de type Harvard et un mode d'adressage particulier avec un fonctionnement RISC (Reduce Instruction Set Computer) caractérisée par des 'instructions plus simple et plus rapides.

Les ASICS ce sont des processeurs plutôt spécifiques à des applications avec des fonctions sur mesure. Les FPGA sont dotés de cellules logiques et de bascules logiques que nous pouvons connecter ou configurer suivant notre application.

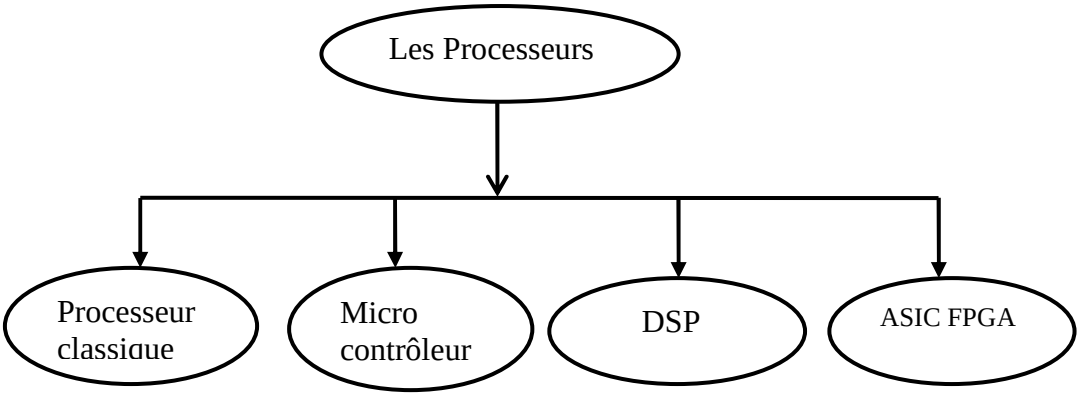


Fig.2 Classification des processeurs.

ASIC: Application Specific Integrated Circuit

FPGA : Field Programmable Gate Array

3 Principe de fonctionnement d'un processeur.

3.1 Introduction.

Un microprocesseur (processeur ou CPU Central Processing Unit) exécute des instructions suivant la cadence d'une horloge. Ces instructions sont écrites soit en assembleur soit avec des langages abstraits traduits par des compilateurs et des interpréteurs. Il est constitué

principalement d'une unité arithmétique et logique UAL, d'un décodeur d'instructions, d'un séquenceur et de registres

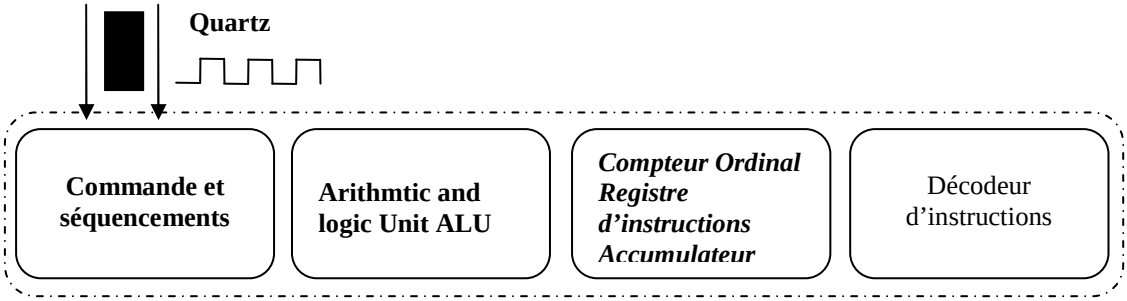


Fig.3 Les sections principales d'un processeur.

3.2 Fonctionnement d'un processeur.

Le microprocesseur connaît à l'aide de son compteur ordinal **CO** l'adresse de la cellule mémoire a laquelle il doit accéder, en utilisant le registre adresse **RA** et le registre mémoire **RM**. Le code de l'instruction est ramené dans le registre d'instruction **RI** et l'unité de décodage a l'aide du séquenceur traduit l'instruction en des commandes électrique tel qu'une addition (groupe de circuits logiques appelé additionneur contenu dans l'Unité Arithmétique et Logique UAL). Enfin l'unité d'exécution exécute 'instruction.

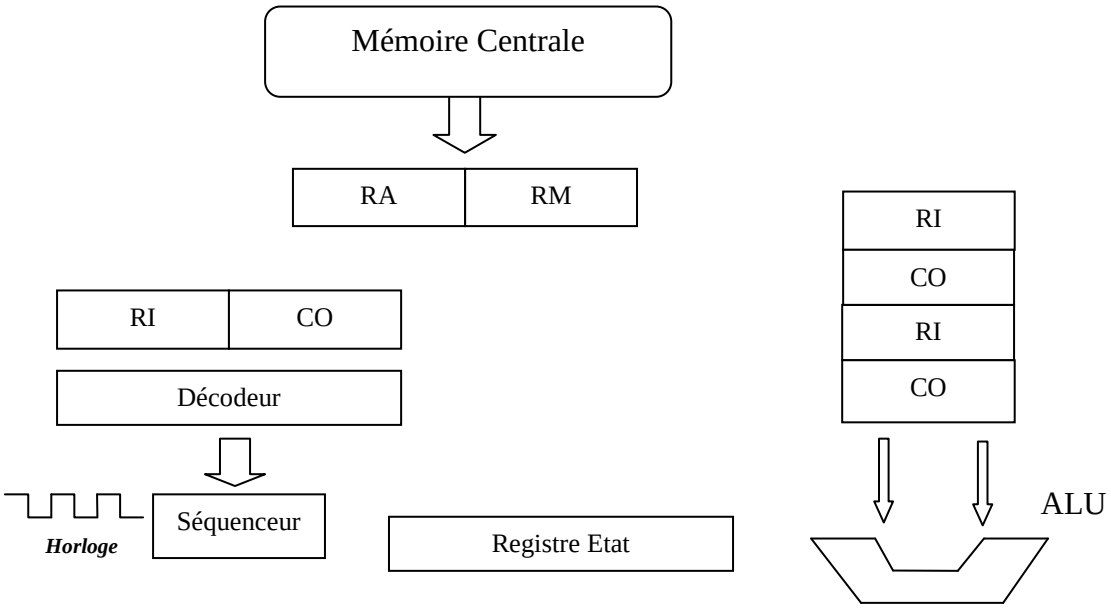


Fig.4 Exécution d'une instruction.

3.3 Exemple de fonctionnement d'un processeur.

La figure ci-dessous illustre le cas d'une opération avec une instruction et un opérande

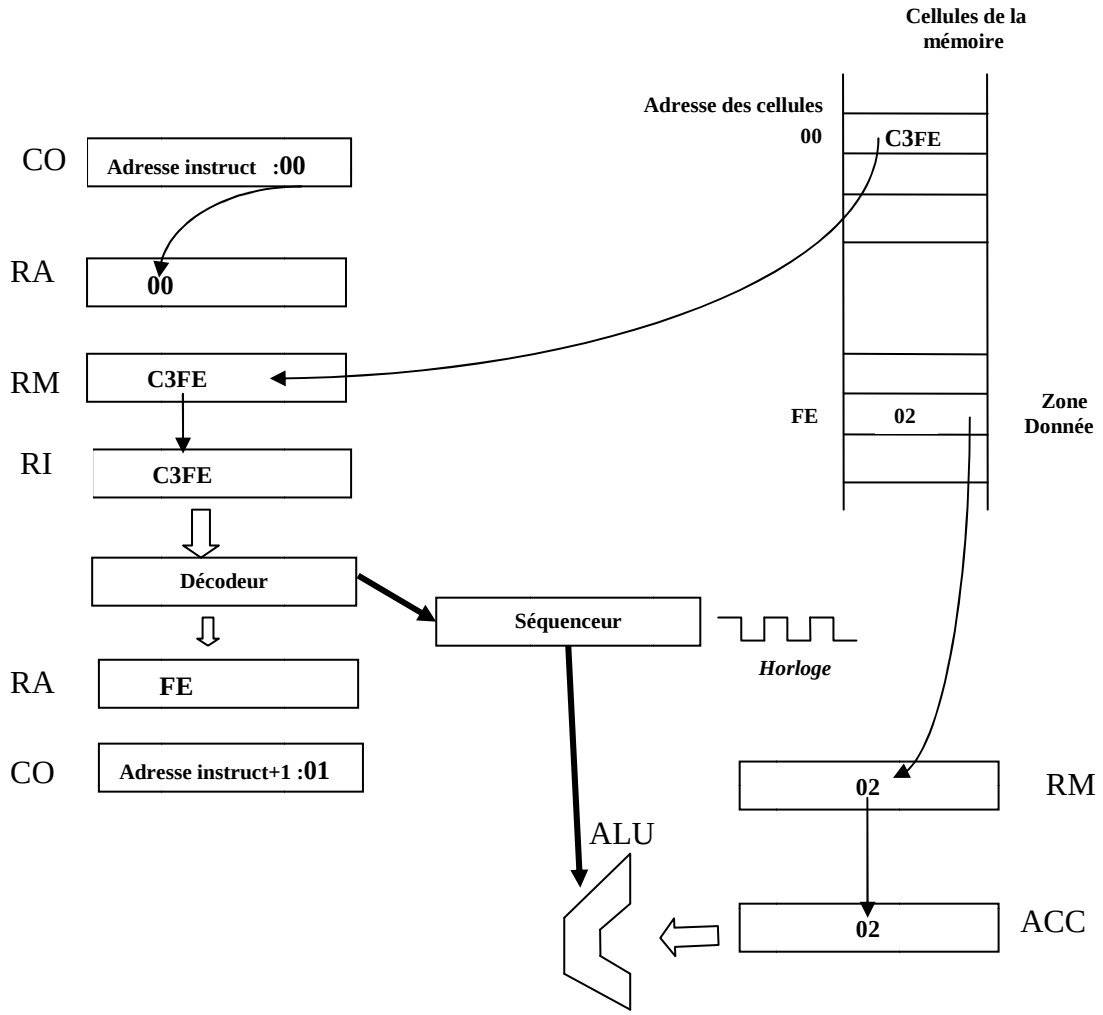


Fig.5 Exécution d'une opération dans un microprocesseur.

Le cas du traitement d'une opération quelconque qui utilise un seule opérande est illustré à la figure 5. Le tout sera cadencé à la vitesse de l'horloge.

Au début le compteur ordinal CO connait l'adresse de l'instruction à exécuter, cette adresse sera placée dans le registre d'adresse RA. Ensuite l'instruction dont l'adresse se trouve dans RA est placée dans le registre mémoire RM.

Notons que l'instruction contient le type de l'opération et l'adresse de l'opérande, cette instruction sera donc ensuite transférée du registre RM au registre d'instruction RI. Le décodeur se charge ensuite de déterminer l'adresse de l'opérande et le type d'opération à exécuter. L'adresse ou se trouve l'opérande est transférée vers le registre RA, le séquenceur prend connaissance de l'opération à exécuter et le CO s'incrémentera automatiquement pour

pointer la prochaine instruction. Cette phase est appelée le cycle de recherche. Au cours du cycle d'exécution l'opérande est transféré dans le registre RM puis dans l'accumulateur et enfin le séquenceur à l'aide de l'UAL exécute l'opération.

3.4 Les Bus.

Dans un microprocesseur les bus peuvent être divisés en trois catégories :

Le Bus de donnée sert à transporter les données ou les instructions du programme.

Le Bus d'adresse indique l'adresse de la cellule mémoire visée ou du dispositif d'entrée sortie.

Le Bus de contrôle transmet par exemple les ordres de lecture ou d'écriture.

4 Structure Von Neumann et Harvard.

4.1. Structure Von Neumann.

L'architecture Von Neumann possède une seule mémoire pour les données et les programmes. Dans ce type de conception les opérations sont réalisées d'une manière séquentielle le processeur à besoin de deux cycles horloge pour compléter une instruction. La figure 6 montre une architecture simplifiée d'un processeur Von Neumann.

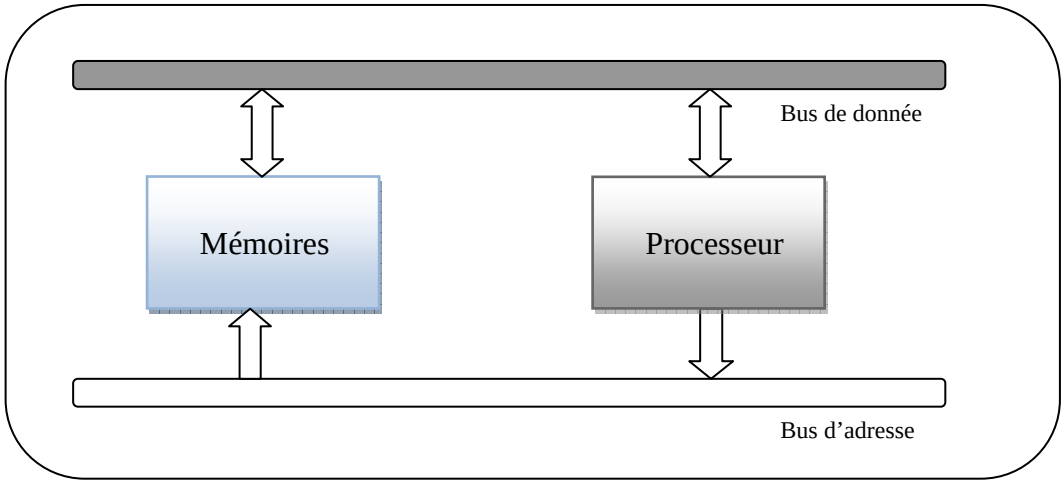


Fig.6 Architecture simplifie Von Neumann.

Pour aller plus vite une des solutions serait d'augmenter la vitesse du processeur nous pouvons exécuter plus rapidement les instructions cependant il faut tenir compte de la dissipation de la température.

4.2 Structure Harvard.

Dans l'architecture Harvard il existe une mémoire programme et une mémoire donnée avec chacune son accès Bus. Ce type de structure nous permet d'exécuter les instructions en

parallèle. En général la mémoire programme contient le code du programme alors que la mémoire donnée contient les données d'entrée par exemple.

Ce type d'architecture nécessite deux fois plus de broches sur le boitier ceci rend la puce beaucoup plus chère. Par souci d'économie, certains processeurs dispose d'une structure Harvard modifiée dans cette dernière la mémoire donnée et la mémoire programme sont toujours séparées avec des Bus Données et des bus Adresses différents. Sauf que la puce ne possède au niveau du boitier que d'un bus donnée et d'un Bus adresse. La communication entre intérieur et extérieur se fera donc à l'aide d'un multiplexage temporel dans la puce.

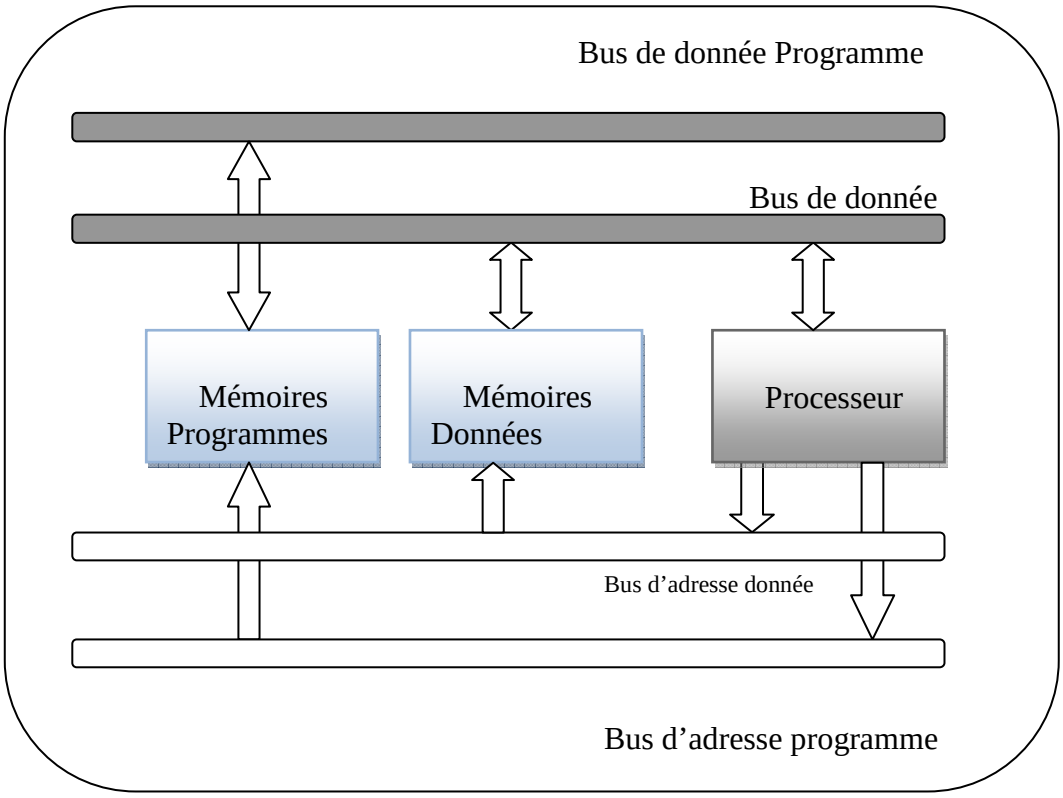


Fig.7 exemple d'une architecture type Harvard.

Il existe aussi l'architecture super Harvard mise au point par la société Analog Device et implémenté dans leur ADSP-2106x SHARC. La particularité de cette architecture est d'une part sa mémoire cache « instruction cache », qui permet d'optimiser encore plus le parallélisme et d'autre part son contrôleur d'entrée sortie « I/O controller » pour la communication à très grande vitesse comme le montre la figure 8.

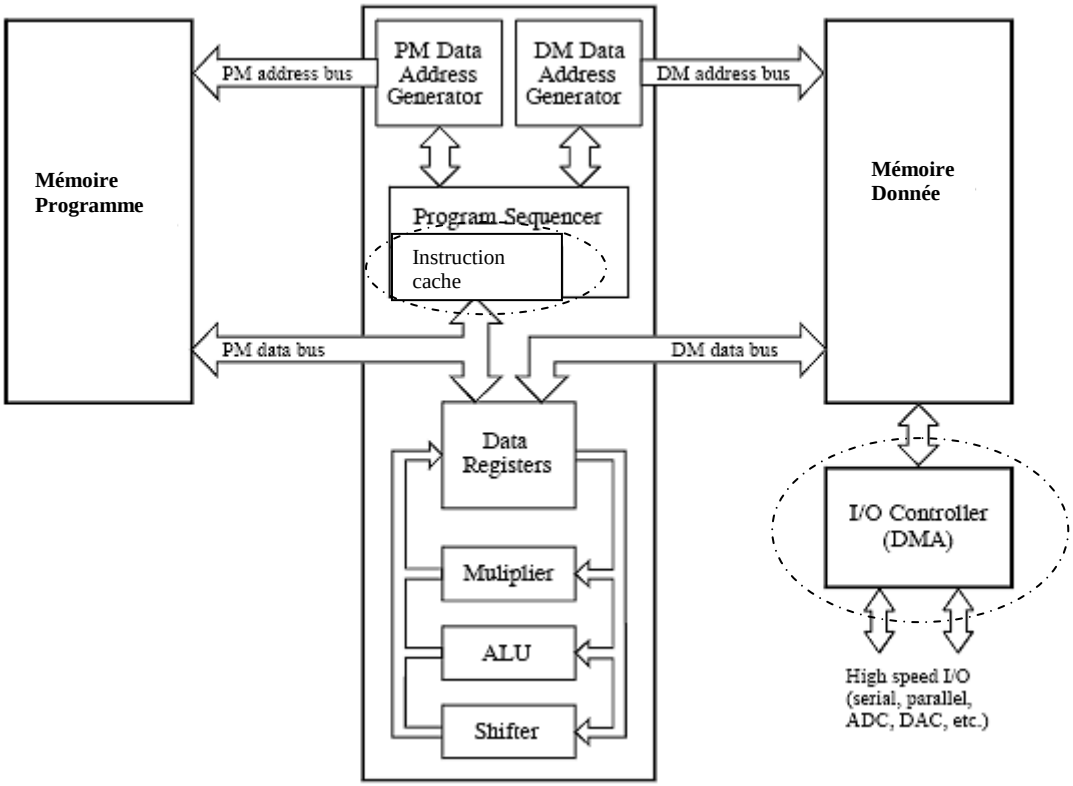


Fig.8 Schéma block du DSP SHARC d'Analog Devices.

Avec cette modification nous pouvons réaliser une multiplication en un cycle horloge en plaçant par exemple la donnée dans la mémoire donnée les coefficients dans la mémoire programme et l'instruction dans le cache.

5. Différents modes de fonctionnement des processeurs.

5.1. Principe de fonctionnement du Pipeline.

L'intérêt d'un Pipeline est d'optimisé la prise en charge des instructions, pour gagner du temps lors de l'exécution d'une routine, formé par plusieurs instructions. L'idée est d'occupé en même temps toutes les unités d'exécutions du processeur. L'exécution de ces instruction en parallèle permet de réduire le temps de calcul d'un ensemble d'instructions destinés à être traitées l'une à la suite de l'autre et non pas le temps nécessaire à l'exécution d'une instruction qui restera toujours d'un cycle horloge par exemple. Autrement dit toutes les instructions seront traitées ensembles mais chacune à une étape différente de l'autre. Un pipeline optimise les ressources du processeur

Une instruction est formée par plusieurs étapes on va supposer dans notre cas quelle est formée de cinq étapes. Notre Pipe line sera donc formé des cinq étages suivants :

- Recherche (Re): charge l'instruction á exécuter depuis la mémoire dans le registre d'Instruction RI

- Décodage (De): décode l'instruction.
- Exécution (EX): exécute l'instruction (par l'unité arithmétique et logique) .
- Mémoire (AM) : Range la donnée dans une mémoire ou charge la donnée d'une Mémoire.
- Ecriture : (EC) : Ecriture des résultats provenant de l'unité arithmétique et logique dans les registres appropriés appelés encore « *write back* ».

La figure 9 montre le cas du séquençage des instructions dans un processeur sans Pipeline. Notre instruction va donc passer simultanément par Cinq étapes, à la première étape l'instruction est chargé depuis la mémoire, puis elle sera décodée a la deuxième étape. A La troisième étape elle sera exécutée par l'unité arithmétique et logique. A la quatrième étape l'instruction sera rangée ou chargé depuis la mémoire vers un registre si c'est nécessaire enfin à la cinquième étape le résultat sera placé dans le registre de destination.

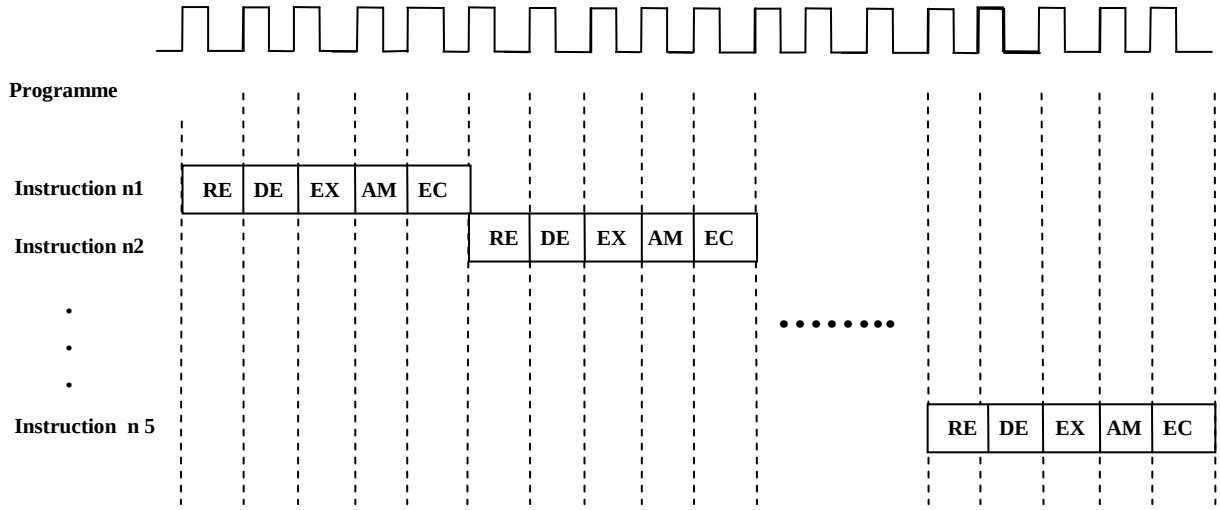


Fig.9 Séquençage des instructions dans un processeur sans Pipeline.

Il faut donc 25 cycles d'horloge pour exécuter les cinq instructions.

La figure 10 montre le gain obtenu dans le cas d'un processeur avec Pipeline. Notons que l'exécution des cinq instructions à nécessité 9 cycles horloge donc un gain de 16 cycles par rapport au processeur sans pipeline (25 cycles). Nous remarquerons aussi que les instructions sont décalées d'un cycle les unes par rapport aux autres de façon à arrivé à ce que les ressources du processeur soient toutes actives en même temps comme le montre la figure 10 au cinquième coup de l'horloge.

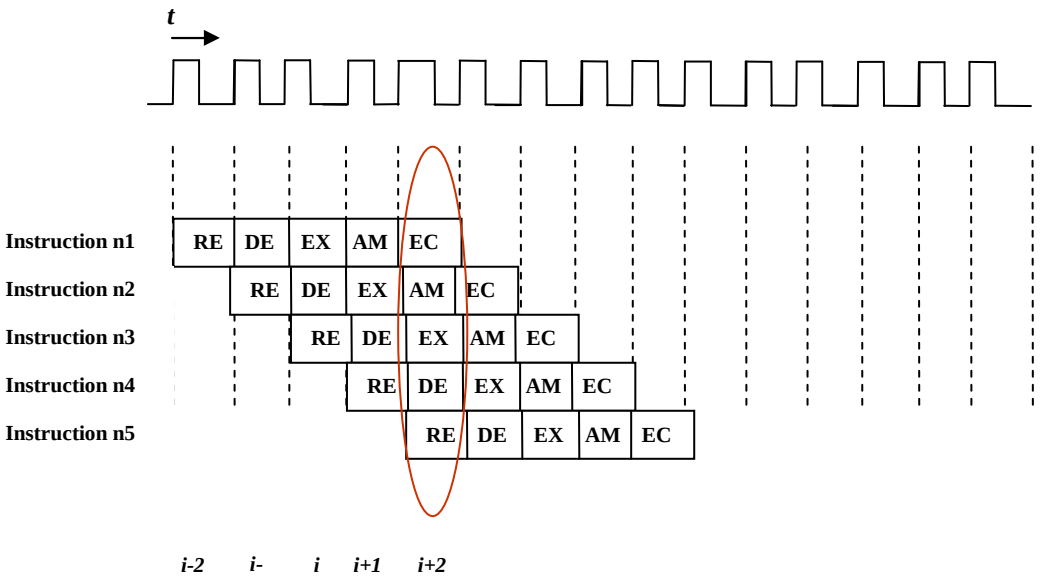


Fig.10 Une illustration du concept de Pipeline .

En insérant des registres tampons au niveau de chaque unité a l'intérieure du processeur, ce dernier pourra contenir plusieurs instructions chacune à une étape. À t=5 toutes les unités du processeur sont sollicitées. Dans le cas du DSP TMS320C25 de Texas Instrument plusieurs registres sont sollicités en effet un premier compteur tient l'adresse de la prochaine instruction à être rechercher, le registre d'instruction (RI) tient l'instruction qui va être exécuté, le registre d'instruction (RIQ) garde l'instruction à être exécuté dans le cas ou l'instruction du RI et encore en exécution. Pendant l'exécution de l'instruction du RI, l'instruction du RIQ est transférée au RI pour exécution.

5.2. Les aléas de fonctionnement d'un pipeline.

Bien qu'utile Les pipelines donnent lieux à de nouveaux problèmes, surtout d'interdépendance entre les différents étages du pipeline nous citons ci-dessus trois aleas.

5.2.1. Les Aléas de Donnée.

Ce disfonctionnement arrive dans le cas ou une instruction à besoin d'une donnée qui n'est pas disponible car elle n'a pas été encore traité par une autre instruction ou qu'elle a été perdu car écrasé par une autre donnée par exemple. Cet aléa peut être évité en ajoutant un chemin de donnée dans le cas où la donnée a été écrasée ou sinon retardé l'instruction suivante d'un cycle ou plus jusqu'à ce que la donnée soit disponible.

5.2.2. Les Aléas structurels.

Cet aléa apparait lorsque deux instructions ont besoin de la même ressource du processeur en même temps deux solutions se présentent soit nous retardons l'instruction qui suit d'un cycle horloge soit nous dédoublons la ressource matérielle. Ce conflit arrive surtout pour les mémoires où les instructions nécessitent souvent des accès mémoires. Cet alea est résolu en utilisant deux bus pour accéder aux données et aux opérations séparément.

5.2.3 Les Aléas de contrôle.

Les aléas de contrôle se produisent lorsque l'instruction suivante dépend du résultat d'une instruction pas encore connu. Ils se forment par exemple quand une instruction de type branchement est prise en compte et la condition de branchement n'est pas encore connue lors du chargement de l'instruction suivante. Pour y remédier il faut que les instructions d'après ne soient pas exécutées. Certains constructeurs utilisent des branchements avec retard ou sinon utilisent la prédiction des branchements à l'aide d'algorithmes de prédictions adaptés.

5.3 Mode super scalaire.

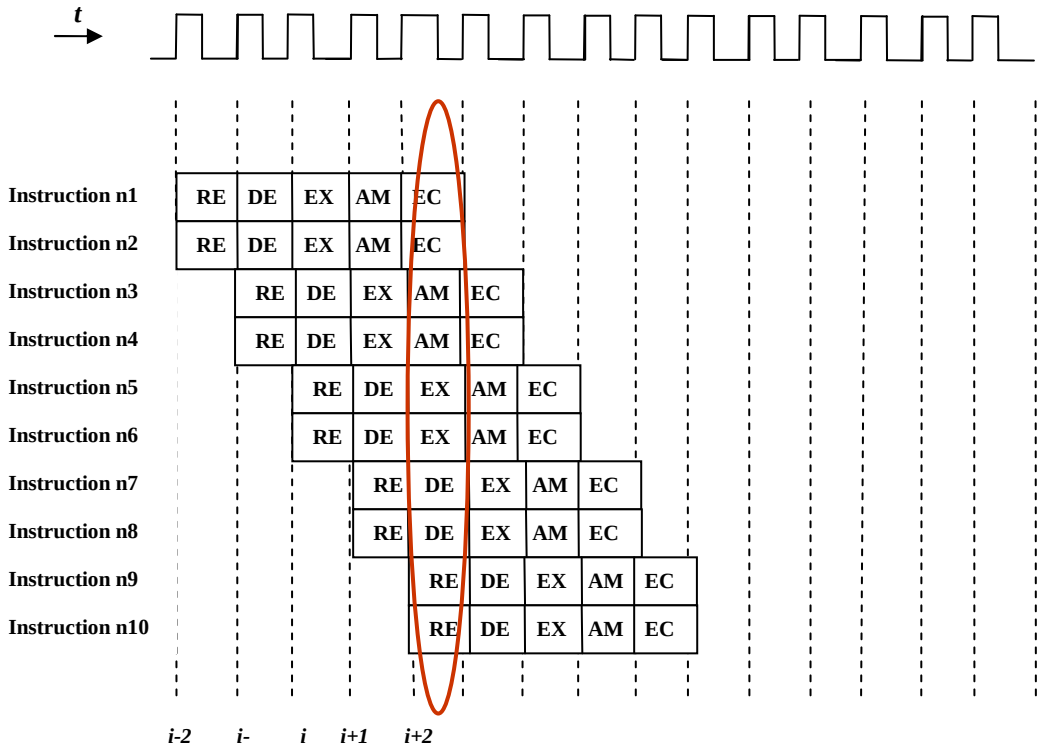


Fig.11 Une illustration du concept de super scalaire.

En multipliant le nombre d'unité dans le processeur, ce dernier pourra exécuter plusieurs instructions en même temps. La figure 11 illustre le cas d'un processeur super scalaire d'ordre 2 il n'a fallu que 9 cycles horloge pour exécuter toutes les instructions.

6 Format de calcul des DSP.

Le codage en virgule fixe est le format le plus rependu en effet ce type de format demande une architecture moins complexe et une consommation inférieure d'énergie. Sauf qu'ils représentent moins bien les nombres avec une grande dynamique. Cependant les processeurs sur 24 bits ou plus arrivent à atteindre une précision satisfaisante mais consomment beaucoup plus de cycle horloge.

Le codage en virgule flottante est meilleur dans le cas d'algorithme nécessitant une puissance de calcul importante et pour les signaux avec une grande dynamique tel que des systèmes avec des variables qui fluctuent énormément (signaux issus des radars, des sonars, …). Les processeurs à virgule fixe sont préférés dans le cas des produits à grande consommation ou le prix de revient est un facteur primordial.

6.1 Codage en virgule fixe.

Pour le codage en virgule fixe les données sont représentées comme des nombres fractionnaires à virgule fixe en complément à deux. La représentation des nombres est comprise entre -1 et $1-2^{-B}$ ou B est le nombre de bits utilisé pour représenter le nombre. La représentation la plus commune pour les DSP est le format *Q15*, qui utilise 16 bits un bit de signe et 15 bits fractionnaire.

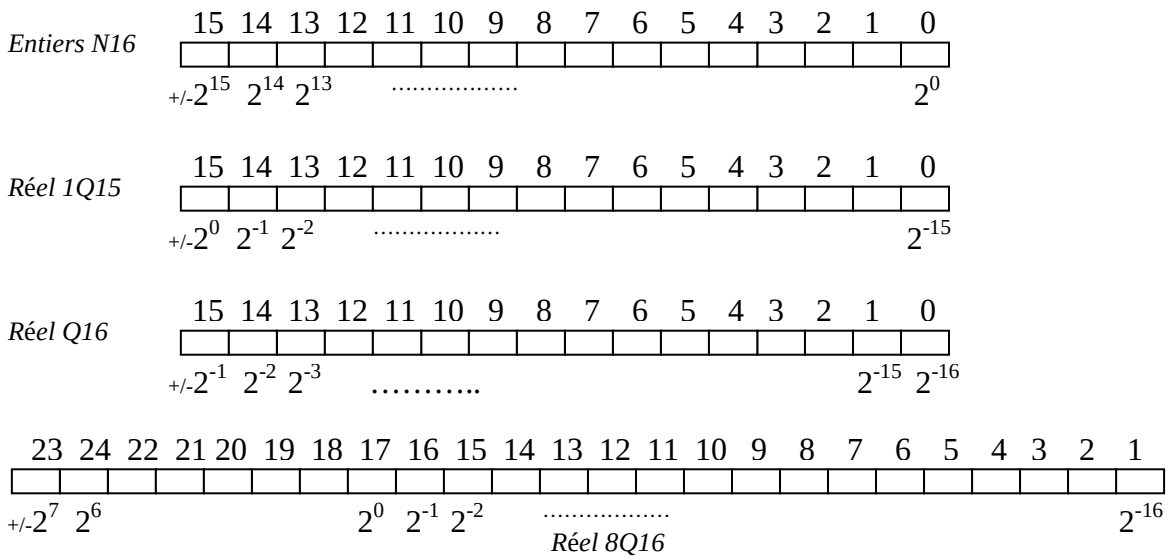


Fig.12 Valeurs signées en complément a deux

Exemple :

Le nombre 0.625(0.5+ 0.125) s'écrit en binaire : 0101 0000 0000 0000 (Réel 1 Q15)

Soit 5 0 0 0 en hexadécimal 5 0 0 0

Le nombre -0.5(-1+0.5 s'écrit en binaire : 1100 0000 0000 0000 (Réel 1 Q15)

Soit 5 0 0 0 en hexadécimal C 0 0 0

6.2 Codage en virgule flottante.

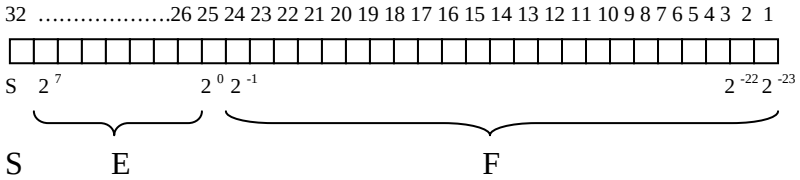


Fig.13 Codage IEEE 754 simple précision 32 bits

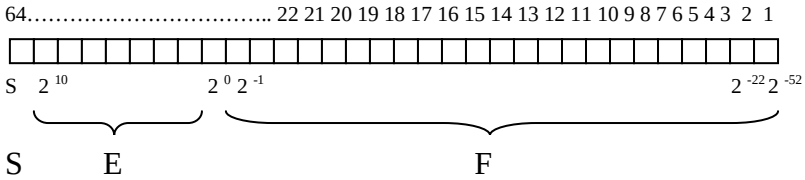


Fig.14 Codage IEEE 754 double précision 64 bits

Le format le plus rependu en virgule flottante est le format IEEE 754. Les figures 13 et 14 donnent les formats simples et doubles précision de la norme IEEE 754. Le nombre décimal X normalisé IEEE d'un nombre en virgule flottante est donné par : $X = (-1^S)(1.F)2^{E-127}$ ou

F est la mantisse, E l'exposant et S le bit de signe.

Exemple :

Le nombre en binaire : 0 10000011 1100…0000 s'écrit 1 x (1.75) x $2^{131-127}$ =1 x 1,75 x 2^4

Le nombre en binaire : 1 00001110 0101……0000 s'écrit -1 x (1.0,3125) x 2^{14-127}

=-1 x 0,3125 x 2^{-113}

Codage flottant IEEE 754 simple précision 32 bits

Largeur des mots : 32 bits

Taille mantisse : 24 bits

Taille exposant : 8 bits

Intervalle mantisse : [1,2-2^{-23}]

Biais exposant: 127

Nbre ordinaire : E + biais Є [1,254], $X = (-1^S)(1.F)2^{E-127}$

Zero : E + biais=0, F=0

Not a number : E + bias =255, F≠0

Infinis : E + bias =255, F=0

Nbre denormalisé: E + bias =255, F≠0, $X = (-1^S)(1.F)2^{-126}$

Min : ≈1.2 x 10^{-38}

Max : ≈3.4 x 10^{38}

Chapitre 2: Les Processeurs Numériques de signaux

1. Introduction.

Un processeur numérique de signal appeler DSP (Digital Signal Processor) dispose de fonctions spéciales pour le traitement numérique du signal. A l'inverse d'un microprocesseur qui ne possède qu'une mémoire à accès simple, le DSP possède une mémoire à accès multiple. Ce qui permet des opérations simultanées d'écriture et de lecture. Ils intègrent un mode d'adressage particulier grâce à plusieurs unités logiques de génération d'adresse configurée avec des registres qui leurs sont propres. Les premiers DSP ont été fabriqué dans les années 1970 dans les domaines militaires et de télécommunication. En 1978 Texas Instrument fabrique le premier DSP pour la synthèse de la parole. Actuellement on trouve les DSP partout dans les applications grand public (téléphone GSM, lecteurs CD, secteur de l'automobile….).

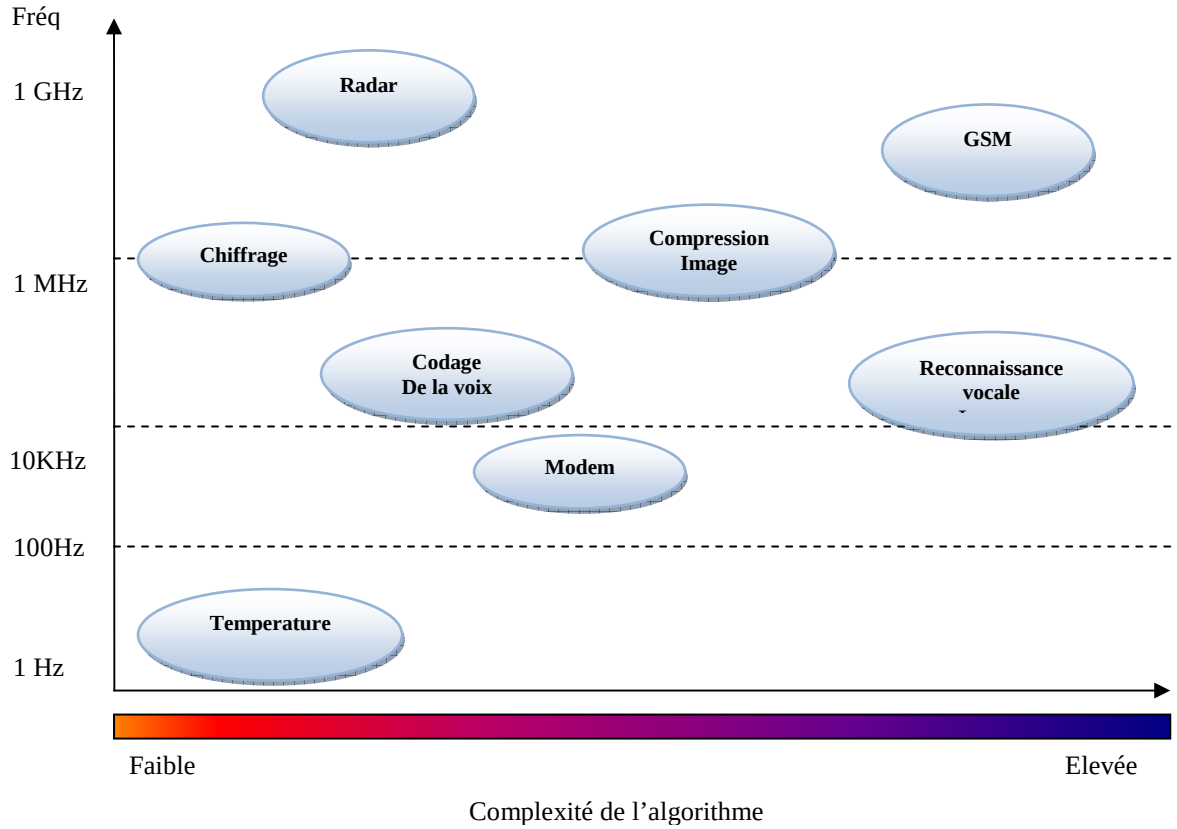


Fig.15. Classement de l'implémentation sur DSP de quelques algorithmes suivant leurs fréquences et leurs complexités.

La figure 15 montre le domaine d'utilisation des DSP qui va du simple algorithme pour le contrôle de gain et l'asservissement de température à des algorithmes très complexes et très gourmands en fréquences d'horloge telque la reconnaissance d'image ou la téléphonie mobile qui nécessite un grand niveau de parallélisme (optimisation du pipeline, gestion de la mémoire…)

2 Filtrage numérique avec un DSP.

2.1 Chaîne typique d un système de traitement numérique du signal.

Une chaîne d'acquisition avec un DSP comprend en général :

- Un processeur de traitement de signal.
- Un convertisseur analogique numérique.
- Un convertisseur numérique analogique.
- Un bloc de mémoire externe.
- Une horloge externe

La figure 16 montre un Schéma bloc d'une carte typique de traitement numérique de signal à base de DSP.

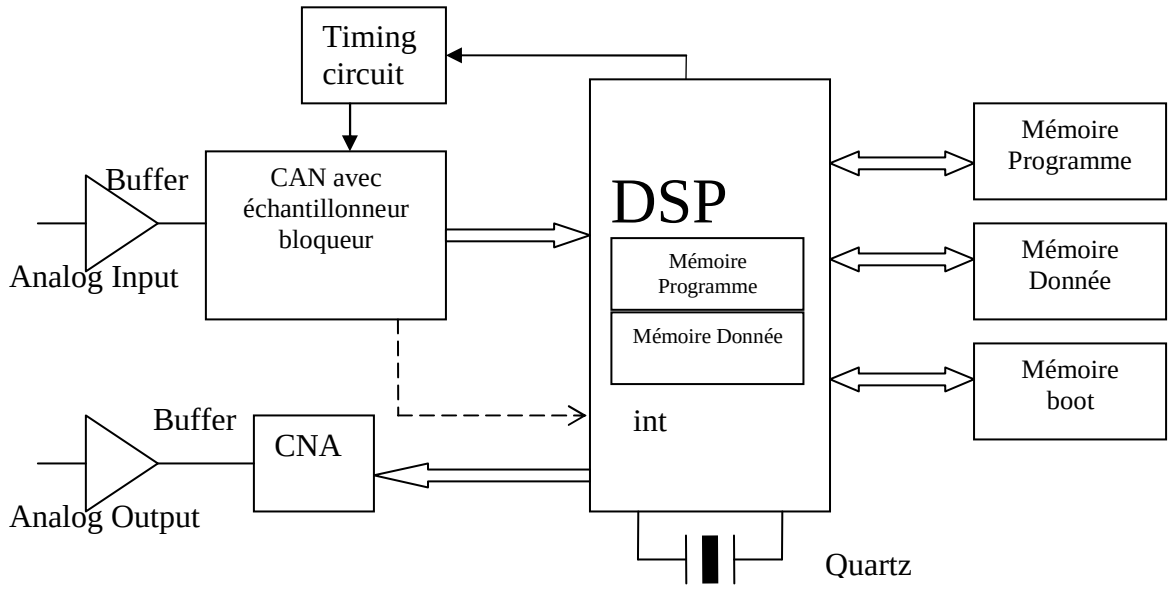


Fig.16 Schéma bloc d'une carte à base de DSP.

2.2. Architecture générale d'un DSP.

La figure 17 montre l'architecture de base d'un DSP, cette dernière comprend une unité de traitement et de calcul adapté au traitement numérique du signal, des Bloc de mémoire internes plus un cache, des bus interne pour la communication des différents éléments entre

eux , une unité de communication pour l'accès aux données externes et éventuellement à d'autres unités de calcul externe.

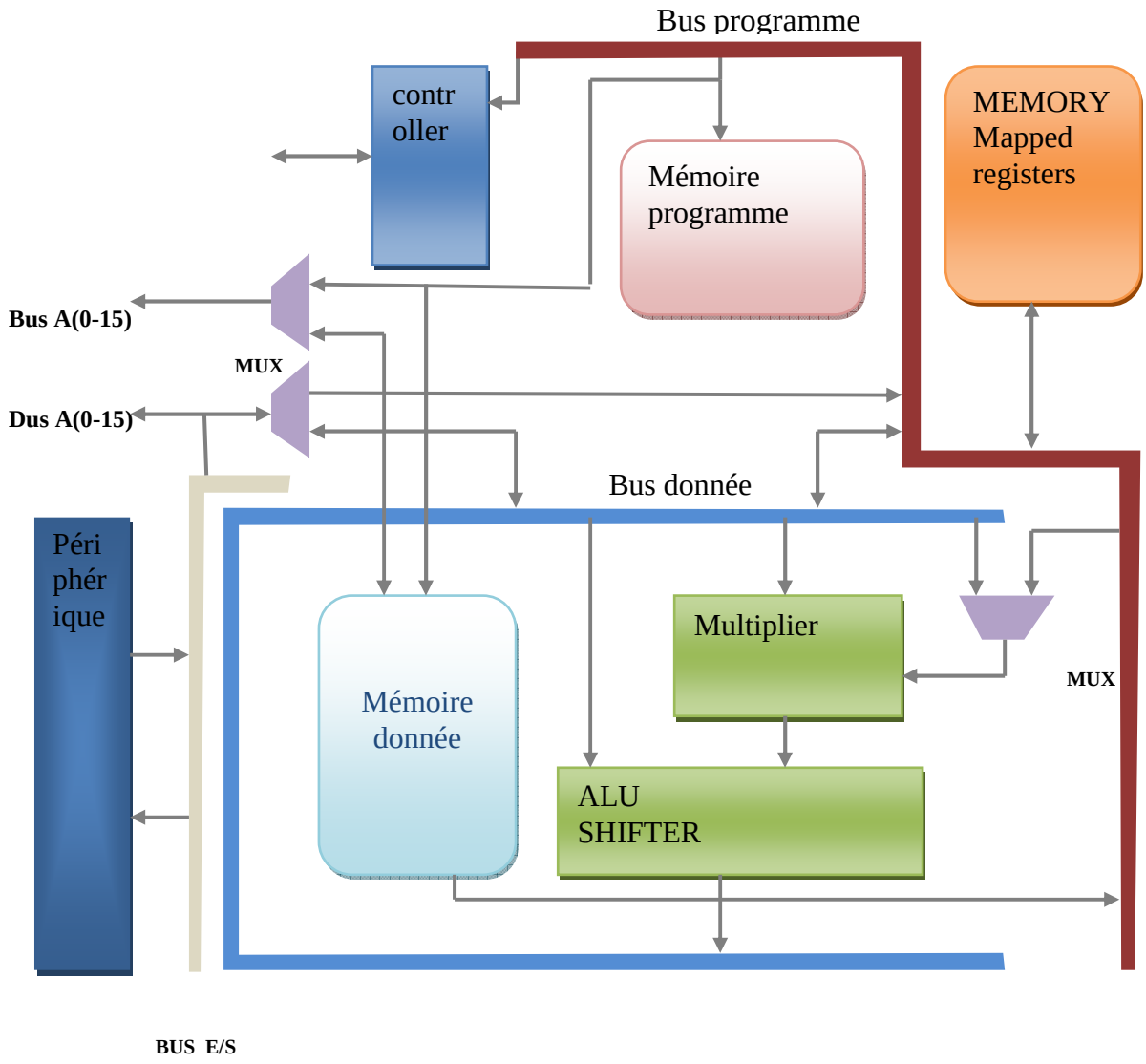


Figure17 Schéma Bloc d'un DSP.

3 Fonctionnement d'un DSP cas de l'ADSP2100

Le traitement numérique du signal fait appel à des opérations de multiplications, d'additions et d'accès mémoire répétitive. Ceci nécessite une architecture particulière que ni les microprocesseurs ni les microcontrôleurs ne proposent. Cette architecture doit intégrer la notion de parallélisme telque un pipeline, un accès rapide aux mémoires, prévoir donc comme le montre l'architecture de la figure 18, une architecture Harvard **(1)**, des mémoires caches**(2)**, des unités d'exécution adapté aux traitements numériques du signal telque un multiplieur additionneur (MAC) **(3)**, une FFT, un adressage particulier **(4)**. La figure suivante montre le cas de l'architecture d'un DSP ADSP2100 d'analog device.

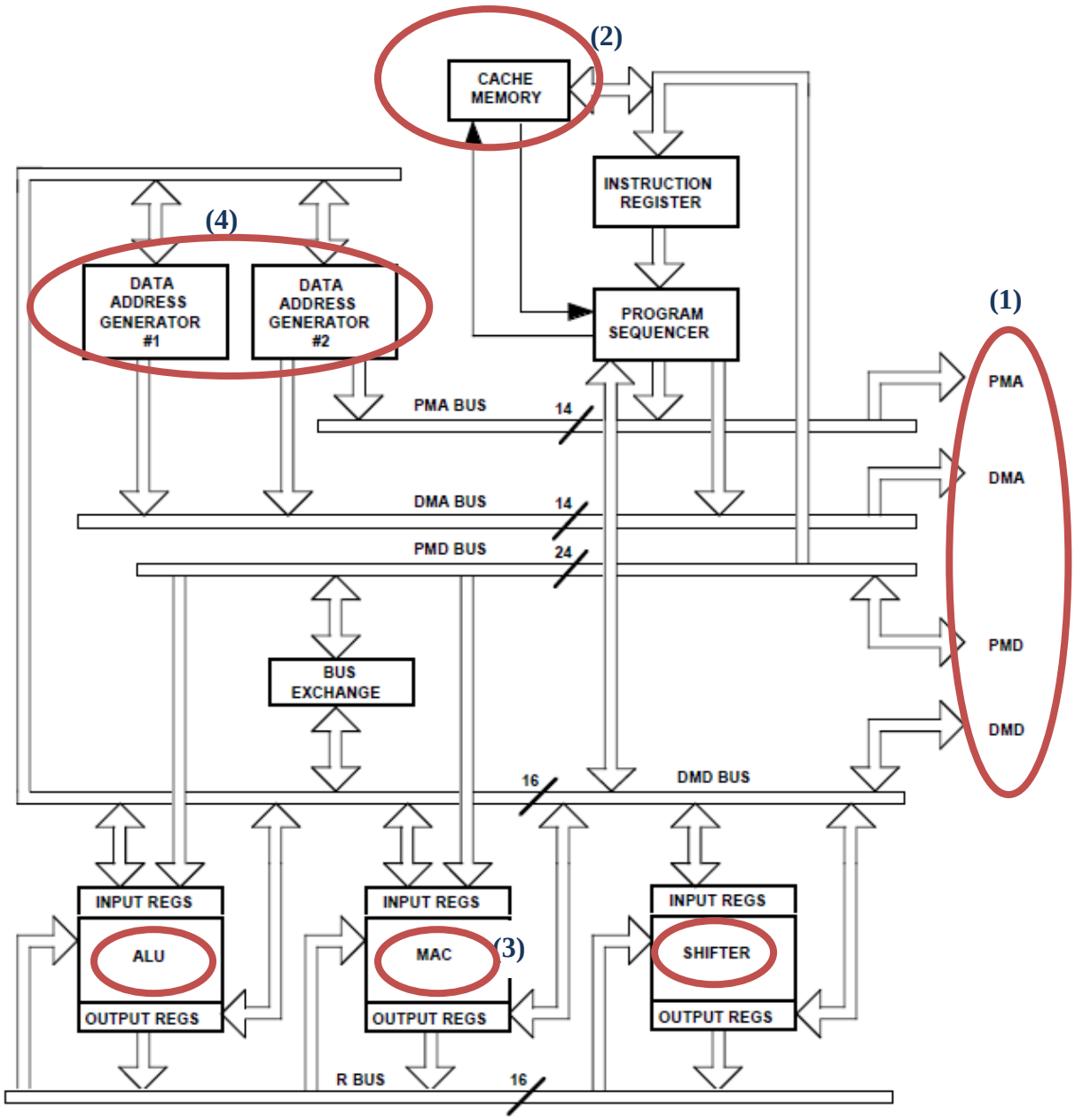


Fig 18 architecture de l'adsp21xx.

3.1 La mémoire Cache.

Dans le but d'accroitre la vitesse d'exécution d'un programme les DSP disposent de mémoires internes (ROM et RAM). Pour gagner encore plus des cycles horloges et donc d'économiser de l'énergie les processeurs de signaux ont aussi des mémoires cache qui stockent les phases répétitives d'un programme.

Nous donnons ci dessous les mnémoniques des registres de l'unité arithmétique et logique ALU, des registres du multiplieur accumulateur MAC, des registres de l'unité de décalage SHIFTER, des registres des pointeurs, des registres des compteurs, des registres d'état et des registres des ports série de la famille adsp21xx.

Mnemonic	*Definition*
AX0, AX1, AY0, AY1	ALU inputs
AR	ALU result
AF	ALU feedback
MX0, MX1, MY0, MY1	Multiplier inputs
MR0, MR1, MR2	Multiplier result (3 parts)
MF	Multiplier feedback
SI	Shifter input
SE	Shifter exponent
SR0, SR1	Shifter result (2 parts)
SB	Shifter block (for block floating-point format)
PX	PMD-DMD bus exchange
I0 - I7	DAG index registers
M0 - M7	DAG modify registers
L0 - L7	DAG length registers (for circular buffers)
PC	Program counter
CNTR	Counter for loops
ASTAT	Arithmetic status
MSTAT	Mode status
SSTAT	Stack status
IMASK	Interrupt mask
ICNTL	Interrupt control modes
RX0, RX1	Receive data registers (not on ADSP-2100)
TX0, TX1	Transmit data registers (not on ADSP-2100)

3.2. L'unité arithmétique et logique .

L'unité arithmétique et logique de l'adsp2100 permet une multitude de fonction standard logique et arithmétique telque l'addition, la soustraction, la fonction NON, la valeur absolue l'incrémentation et la décrémentation. Comme le montre le schéma bloc de la figure 19, l'ALU possède deux entrées X et Y sur 16 bits et une sortie R sur 16 bits elle accepte aussi la retenu et est capable de générer six signaux état (STATUS).

L'entrée X est liée aux registre AX (AX0 et AX1) ces registres sont connectés au bus DMD en écriture et en lecture et au registre résultat R. L'entrée Y est liée aux registres AY (AY0 et AY1) ces registres sont connectés au bus DMD en écriture et au bus PMD en lecture et au registre résultat AF. Notons que n'importe quel registre associé à l'ALU peut être accéder en lecture et en écriture au cours du même cycle horloge.

3.2.1 Les registres de l'ALU.

AX0, AX1	AY0, AY1	AR
AR	AF	AF
MR0, MR1, MR2		
SR0, SR1		

Notons que MR0, MR1 et MR2 sont les registres résultats du multiplieur

SR0, SR1 sont les registres résultat du shifter

3.2.2 Fonctions standard de l'ALU.

Ci-dessous les fonctions standards de l'unité arithmétique et logique de l'adsp2100:

R = X + Y	Add X and Y operands
R = X + Y + CI	Add X and Y operands and carry-in bit
R = X – Y	Subtract Y from X operand
R = X – Y + CI - 1	Subtract Y from X operand with "borrow"
R = Y – X	Subtract X from Y operand
R = Y – X + CI - 1	Subtract X from Y operand with "borrow"
R = – X	Negate X operand *(twos-complement)*
R = – Y	Negate Y operand *(twos-complement)*
R = Y + 1	Increment Y operand
R = Y – 1	Decrement Y operand
R = PASS X	Pass X operand to result unchanged
R = PASS Y	Pass Y operand to result unchanged
R = 0 *(PASS 0)*	Clear result to zero
R = ABS X	Absolute value of X operand
R = X AND Y	Logical AND of X and Y operands
R = X OR Y	Logical OR of X and Y operands
R = X XOR Y	Logical Exclusive OR of X and Y operands
R = NOT X	Logical NOT of X operand *(ones-complement)*
R = NOT Y	Logical NOT of Y operand *(ones-complement)*

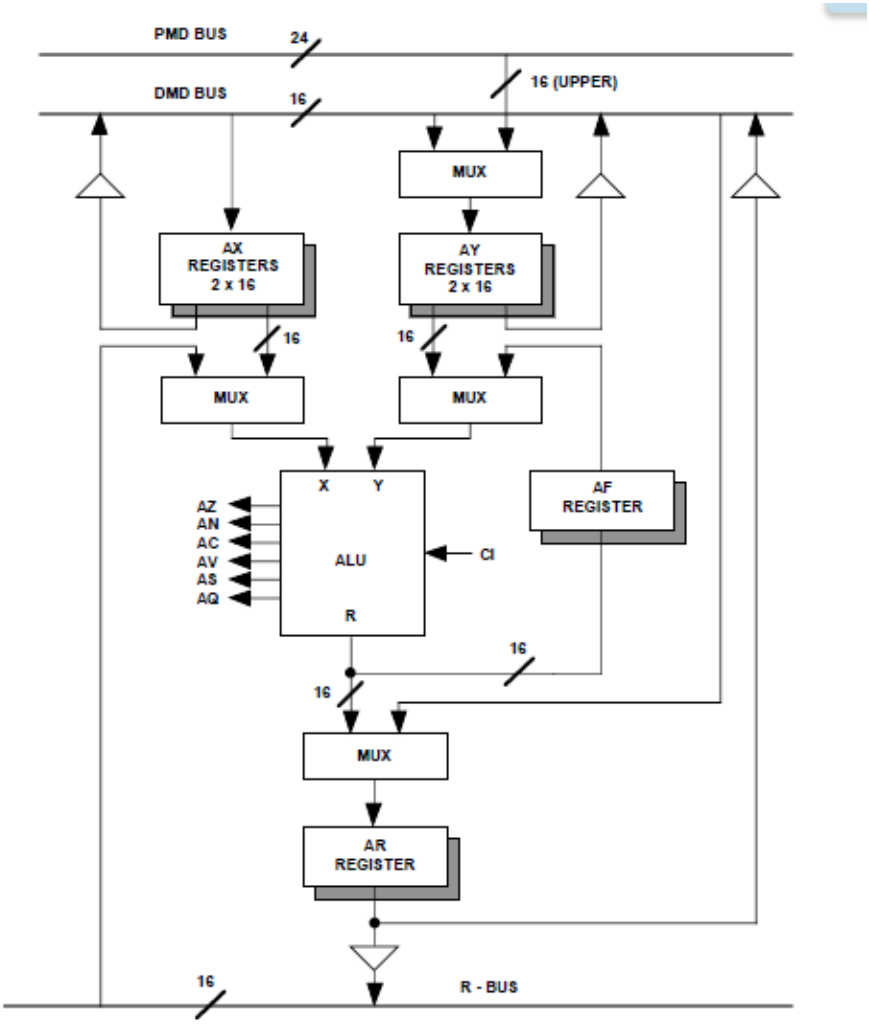


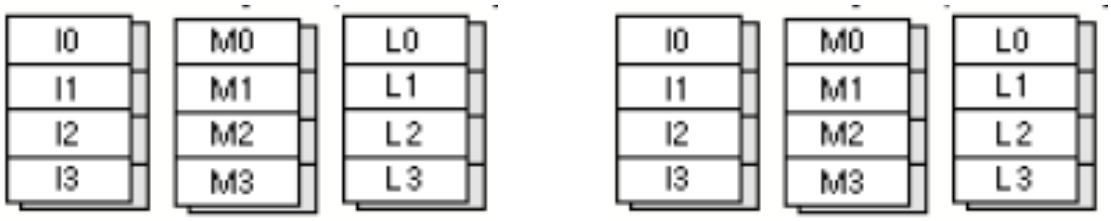
Fig.19 Schéma bloc de l'ALU de l'adsp2100.

3.3 Les différents modes d'adressage.

Les DSP possèdent plusieurs unités logiques de génération d'adresse. Une unité logique une fois paramétré à l'aide des registres appropriés, peut générer toute seule en parallèle avec l'exécution d'une opération arithmétique les adresses nécessaires à l'accès des données.

Dans le cas de la famille ADSP21xx Les registres adresse donnée sont divisés en deux groupes (DAG1et2)

- Le premier groupe DAG1 ne peut pointer que la mémoire donnée DM. Les registres index i0, i1, i2, i3 sont sur 14 bits, les registres pas d'incrémentation, de décrémentation et les longueurs de ces registres sont respectivement m0, m1, m2, m3 et l0, l1, l2, l3



DAG1 DAG2

- Le deuxième groupe de générateur d'adresse est le groupe DAG2. Il peut pointer en même temps sur les mémoires donnée DM et programme PM. Les registres index sont également sur 14 bits i4, i5, i6, i7 et les registres Pas d'incrémentation, de décrémentation et les longueurs de ces registres sont respectivement m4, m5, m6, m7 et l4, l5, l6, l7. La figure 20 montre le diagramme du générateur d'adresse 1 (DAG1).

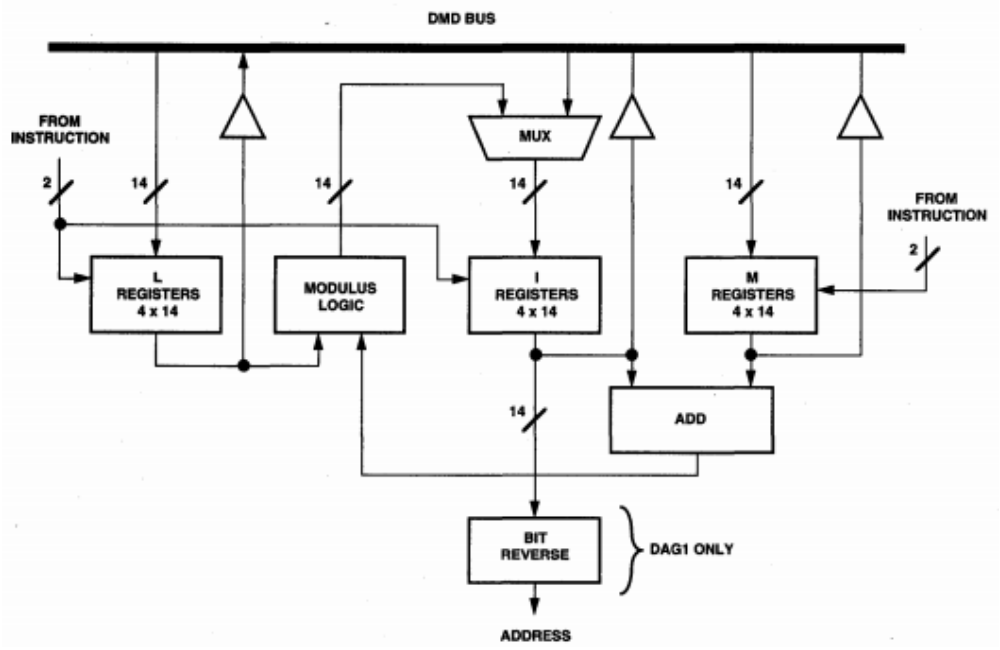


Fig.20 Block Diagramme du Générateur d'adresse 1 (DAG1).

En plus des adressages standards les DSP possèdent des modes d'adressages spécifiques telque l'adressage modulo très apprécié pour le calcul de la covolution ou le bit reverse est particulièrement intéressant dans le calcul de l'algorithme de transformée de fourrier rapide FFT.

Le générateur d'adresse 1 permet d'inverser l'adresse de sortie avec le bit reverse mode (figure20). Ce mode sera activer à l'aide du registre status MSTAT ou en utilisant l'instruction ENA BIT_REV.

MSTAT est un registre de 7 bits qui défini les différents modes opératoires du processeur, comme le montre la figure 21.

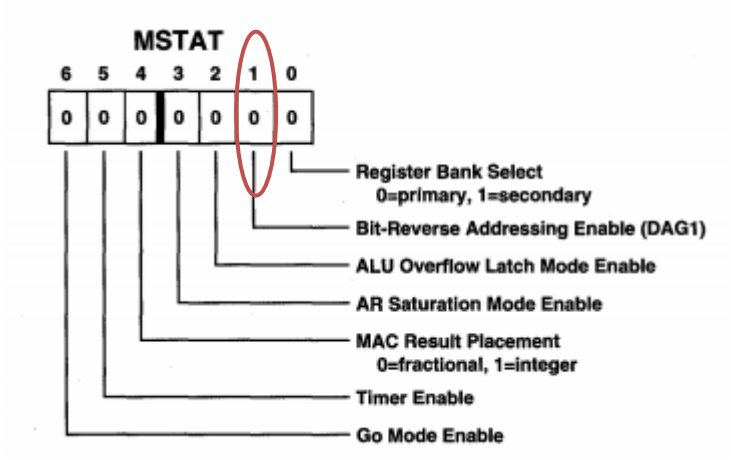


Fig21 Registre MSTAT.

0	0	0	0		0	0	0	0
1	0	0	1		4	1	0	0
2	0	1	0		2	0	1	0
3	0	1	1	→	6	1	1	0
4	1	0	0		1	0	0	1
5	1	0	1		3	1	0	1
6	1	1	0		5	0	1	1
7	1	1	1		7	1	1	1

Fig.22 Cas d'un Bit inversé sur 3 bits.

3.3.1 Adressage standard.

Adressage implicite : lorsque l'instruction ramenée se réfère à un des registres internes du processeur tel que l'accumulateur.

Adressage a registre : lorsque l'instruction ramenée spécifie un registre (Ex INC B).

Adressage immédiat : La donnée sur laquelle porte l'instruction ce trouve dans l'instruction

elle même. (Ex Mov R1, 17)

Adressage direct : L'instruction inclus l'adresse de la cellule mémoire dans laquelle ce trouve l'opérande (Ex Mov R1, M(EA15)).

Adressage indirect : L'instruction désigne un registre dans lequel se trouve l'adresse de l'opérande. (Ex Mov R1, R2).

Adressage indirect avec incrémentation ou décrémentation : on lit ou écrit dans des adresses successives de la mémoire. Il faut ajouter 1 au contenu du registre d'adresse pour lire la cellule mémoire suivante.

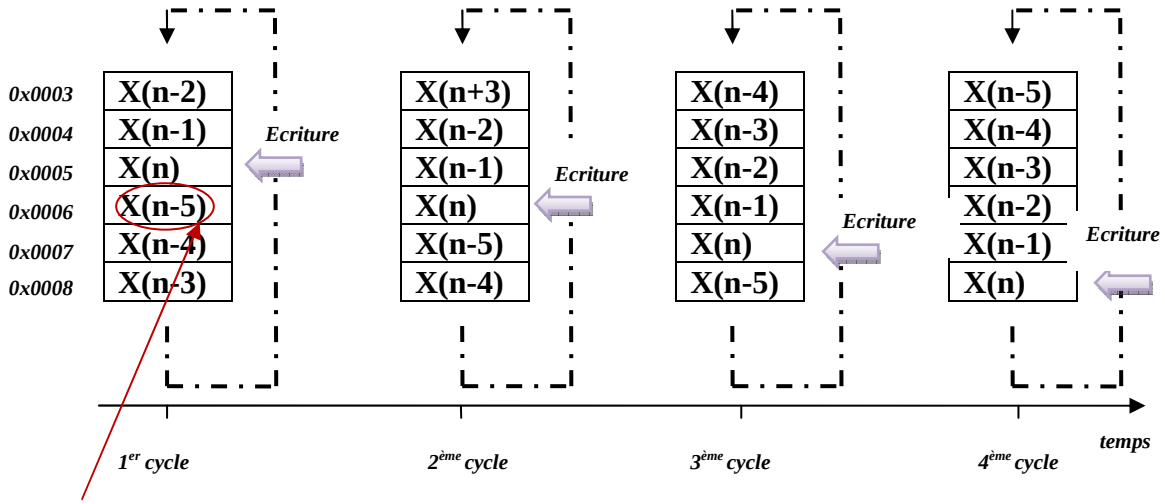
Adressage indirect avec Pré ou post incrémentation ou décrémentation : Le registre est incrémenté ou décrémenté après l'exécution de l'instruction. Très utile lors des calculs répétitifs sur une série de données rangées séquentiellement en mémoire.

Adressage relatif : C'est un adressage indirect par registre plus un déplacement (Ex Mov(R1, M(R2 + 3))).

Adressage indexé : C'est un adressage indirect par registre plus un registre d'index (Ex Mov(R1, M(R2 + R3))).

3.3.2 Adressage circulaire ou modulo.

Les unités logiques de génération d'adresse peuvent générer un adressage circulaire indirect avec pré ou post incrément ce type d'adressage fait gagner beaucoup de temps pour accéder aux opérandes du multiplieur MAC dans le cas d'un filtrage FIR ou IIR.



X(n-5) valeur la plus ancienne

X(n) Ecriture nouvelle donnée

Fig.23 Evolution d'un buffer circulaire.

La figure n°23 montre le cas d'un buffer circulaire de longueur 5

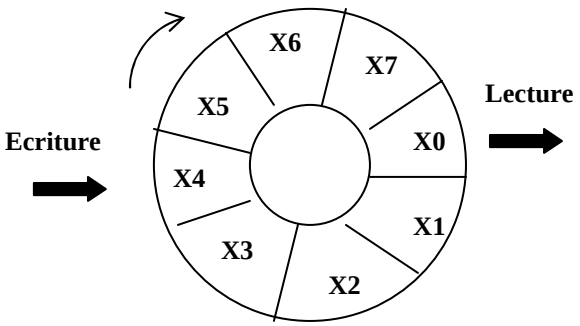


Fig.24 buffer circulaire a 7 éléments.

Il existe deux types de paramétrage du buffer soit l'on donne le départ et la longueur soit le départ et l'arrivée.

Dans le buffer circulaire de la famille ADSP 21XX l'adresse suivante est calculée en respectant la formule ci dessous :

ADR Suivante= (I+M-B) Modulo (L) + B

Ou I : adresse courante

B : Adresse de base

M : pas d'incrément

L : Longueur du buffer

Avec valeur absolue de (M) <L

L'adresse de base d'un buffer circulaire de longueur L doit être 2^n ou multiple de 2^n avec n comprise entre $2^{n-1}<L<2^n$.

3.4 La configuration MAC *Multiply and ACcumulate*.

En utilisant des instructions spécialisées le processeur de signaux à l'aide de son MAC pourra lire en mémoire une donnée, effectuer une multiplication puis une addition et enfin écrire en mémoire le résultat et ceci en un seul cycle d'horloge.

La figure 25 montre une configuration MAC d'analog device famille ADSP21xx avec deux registres de 16 bits pour les données et un registre produit de 32 bits ou sera stocké le résultat de la multiplication. La sortie du registre produit R est connectée à un accumulateur double précision, ou le produit sera accumulé.

Les fonctions standard du MAC.

X*Y	Multiply X and Y operands.
MR+X*Y	Multiply X and Y operands and add result to MR register.
MR–X*Y	Multiply X and Y operands and subtract result from MR register.
0	Clear result (MR) to zero.

Les registres entrés sorties du MAC.

Registres entrée X	Registres entrée Y	Registres sortie R
MX0, MX1	MY0, MY1	MR (MR2, MR1, MR0)
AR	MF	MF
MR0, MR1, MR2		
SR0, SR1		

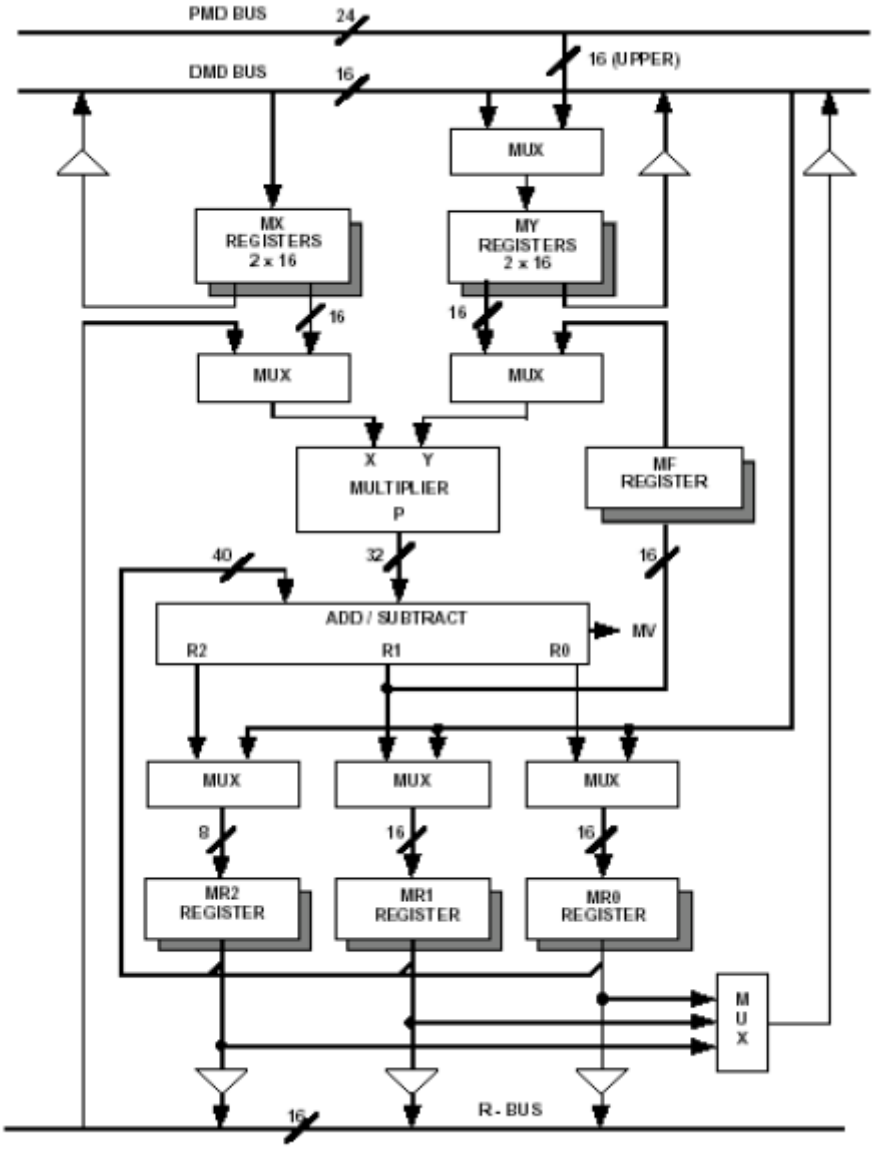


Fig.25 Une Configuration MAC

3.5. Le contrôleur d'interruption.

Les tableaux ci-dessous montrent les vecteurs d'interruption de l'ADSP2101 et de l'ADSP 2181 de la famille ADSP21XX d'analog device. Les interruptions sont classées par ordre de priorité. Notons que le SPORT1 peut être configuré soit comme port série en réception et en transmission soit comme deux interruptions externes IRQ1 et IRQ0. Le DSP dispose aussi d'une interruption externe IRQ2. Les lignes d'interruptions internes sont générées par le

Timer ou les ports séries. Les interruptions peuvent être masquées en utilisant le registre IMASK comme le montre la figure 26 seul le RESET ne peut pas être masqué.

Le registre IMASK est un registre à 6 bits qui valide les interruptions par exemple si IMASK=H#20 ceci veut dire que l'interruption IRQ2 est validée.

Il existe aussi le registre ASTAT sur 8 bits qui défini l'état du processeur tel que la retenue, la saturation du registre de l'ALU ou du registre MAC…

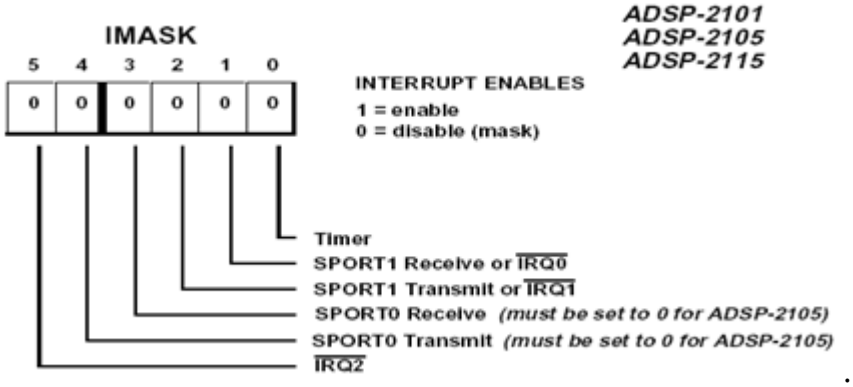


Fig.26 Le registre de masque des interruptions .

Nous pouvons aussi forcer ou effacer les interruptions à l'aide du registre IFC (figure27). En mettant les bits à 1 ou 0 de l'interruption en question.

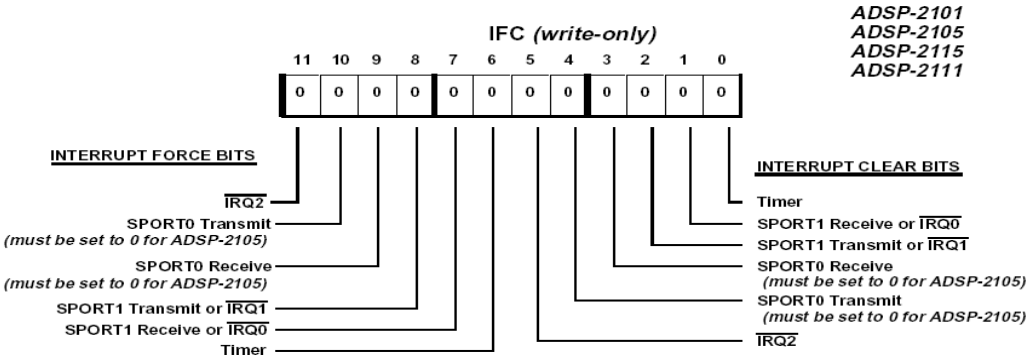


Fig.27 Le registre de forçage des interruptions .

Les interruptions et le Vecteur d'interruption de l'ADSP2101.

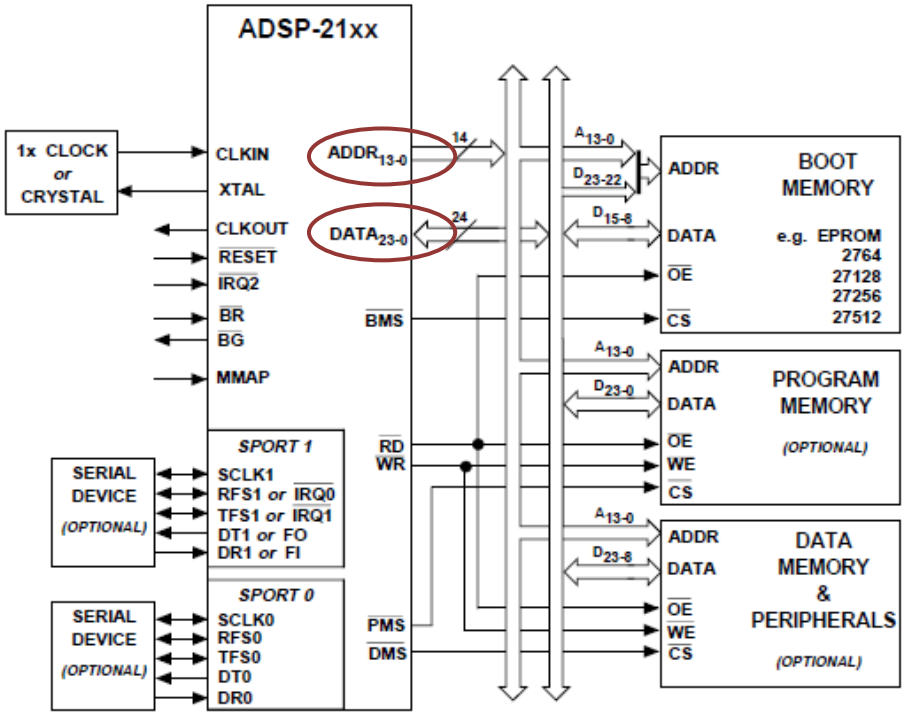
Interrupt Source	*Interrupt Vector Address*
RESET startup	0x0000
IRQ2	0x0004 *(highest priority)*
SPORT0 Transmit	0x0008
SPORT0 Receive	0x000C
SPORT1 Transmit *or* IRQ1	0x0010
SPORT1 Receive *or* IRQ0	0x0014
Timer	0x0018 *(lowest priority)*

Les interruptions et le Vecteur d'interruption de l'ADSP2181.

Interrupt Source	*Interrupt Vector Address*
$\overline{\text{RESET}}$ startup (or powerup w/PUCR=1)	0x0000 *(highest priority)*
Powerdown (non-maskable)	0x002C
$\overline{\text{IRQ2}}$	0x0004
$\overline{\text{IRQL1}}$ (level-sensitive)	0x0008
$\overline{\text{IRQL0}}$ (level-sensitive)	0x000C
SPORT0 Transmit	0x0010
SPORT0 Receive	0x0014
$\overline{\text{IRQE}}$ (edge-sensitive)	0x0018
Byte DMA Interrupt	0x001C
SPORT1 Transmit *or* $\overline{\text{IRQ1}}$	0x0020
SPORT1 Receive *or* $\overline{\text{IRQ0}}$	0x0024
Timer	0x0028 *(lowest priority)*

3.6 Les mémoires.

La figure 28 montre l'interfaçage de l'ADSP avec les différents types de mémoires qu'il supporte. Nous noterons que la famille ADSP21xx a bel et bien une architecture Harvard modifié car il n'y a qu'un seul bus pour les adresses et un seul bus pour les données. Nous remarquerons aussi que les données seront transférer a l'aide d'un bus de 24 bits pour la mémoire programmes et 14 bits pour l'adressage. Alors que ceux de la mémoire donnée seront transférés à l'aide d'un bus de 16 bits pour les données et 14 bits pour les adresses



,

Fig.28 Interfaçage de l'ADSP 21xx avec différent type de mémoire.

3.6.1 Accès direct a la mémoire.

Un port DMA ou Direct Memory Accès permet le transfert des données directement sans passer par le processeur ou les bus interne ce qui permet un accès rapides aux données.

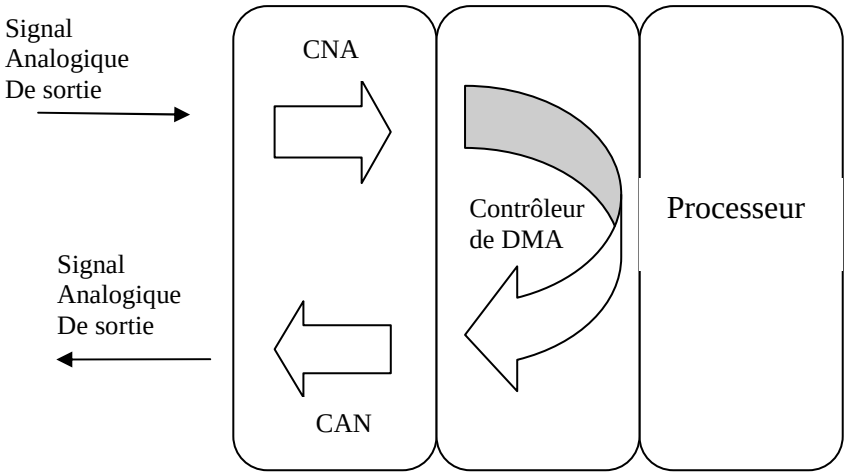


Fig.29 Principe de fonctionnement d'une DMA

Ce transfert de données peut être entre mémoires ou entre mémoire et port série ou port UART….l'ADSP2181 propose deux types de DMA :

- Byte Memory & Byte Memory DMA (BDMA)
- Internal Direct Memory Access (IDMA) Port

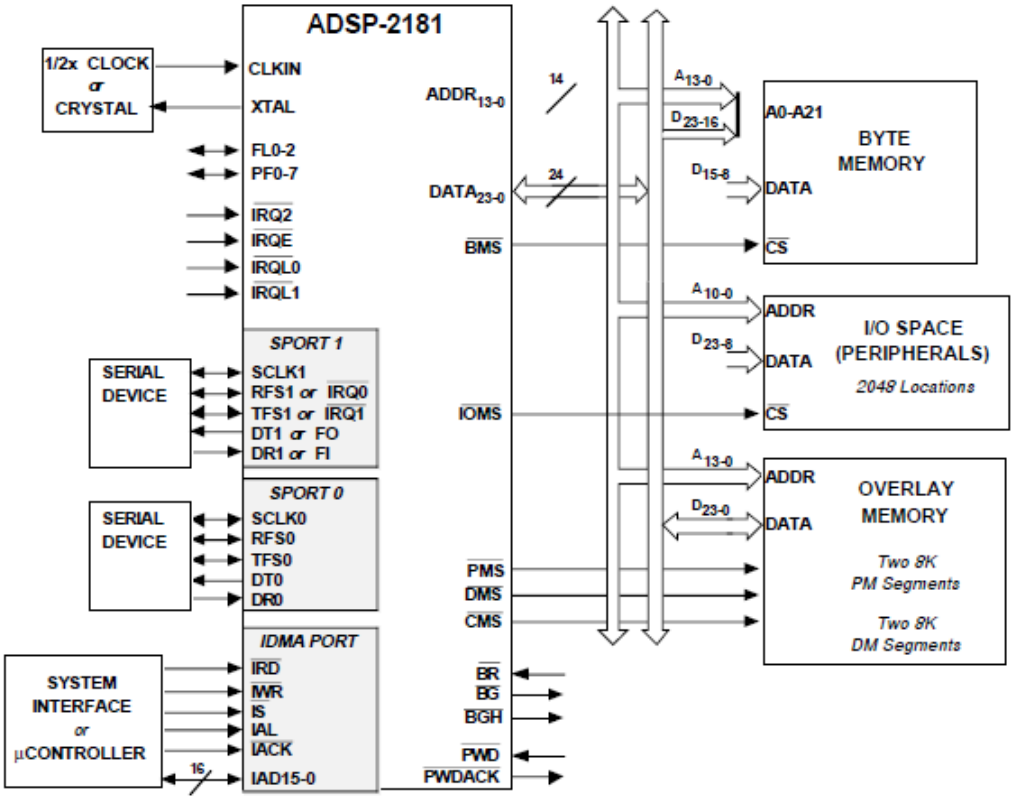


Fig.30 cas de l'ADSP 2181.

La figure 30 montre le cas de l'ADSP2181 avec son IDMA port (Internal Direct Memory Access) ce port DMA permet le transfert des données de la mémoire interne sans sollicité le processeur processeur.

3.7 Port série.

Le port série transmet la donnée sur sa sortie DT la donnée transmise est chargée préalablement a partir d'un registre interne du processeur au registre TX a travers le bus DMD. Une fois la donnée transmise le port série génère une interruption pour signaler qu'il est près pour une nouvelle transmission de donnée. Il reçoit la donnée sur son entrée DR. Une fois la donnée reçue elle est écrite dans le registre RX puis une interruption est générer. Notons que le port série peut recevoir et transmettre en même temps la donnée (en full duplex).

Les ports séries doivent être initialisés avant d'être utilisé à travers des registres de configuration. Ces registres se situent aux adresses 0x3FF3 à 0x3FFA pour le premier port série SPORT0 et aux adresses 0x3FEF a 0x3FF2 pour le deuxième port série SPORT1

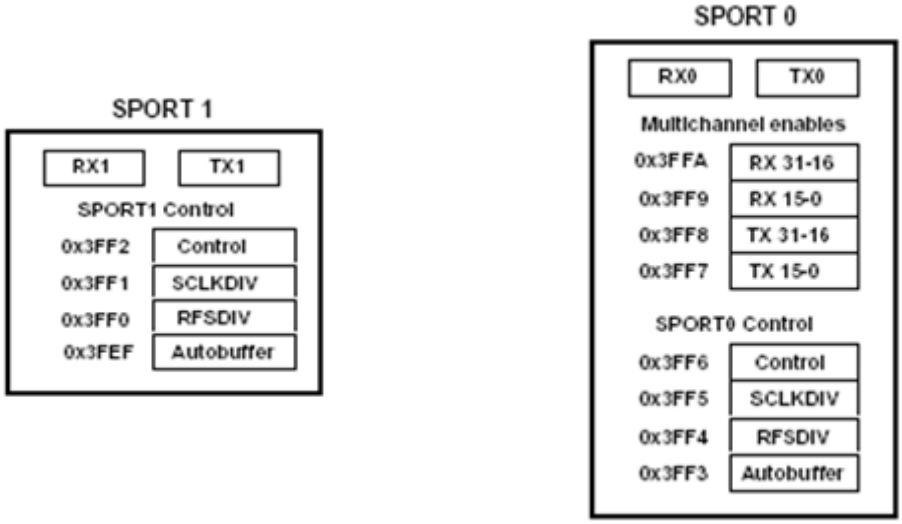


Fig.31 Les ports séries de la famille ADSP21xx.

3.7.1 Exemple de configuration des ports séries de l'ADSP 21xx.

```
{—— Initialisation du SPORT——}
{SPORT1 inits }
AX0 = 0x0017;
DM(0x3FEF) = AX0;                    { SPORT1 en autobuffering}
                                     {TX autobuffer utilise I0 et M0}
                                     {RX autobuffer utilise I1 e M1}
AX0 = 0x280F;
DM(0x3FF2) = AX0;                    { clock serie externe, RFS et TFS}
```

```
                                        {Largeur du mot 16 bits}
{SPORT0 inits}
{Hypothèse: CLKIN à 12.288 MHz. générer en interne}
{SCLK will be 2.048 MHz and framing sync of 8 kHz}
AX0 = 255;
DM (0x3FF4) = AX0;                      {RFSDIV = 256, 256 SCLKs entre}
                                        {Frame syncs: 8 kHz framing}
AX0 = 2;
DM(0x3FF5) = AX0;                       {SCLK = 2.048 MHz}
AX0 = 0x6B27;
DM(0x3FF6) = AX0;                       { SCLK interne, RFS et TFS}
                                        {normal framing, mu-law companding}
                                        {Largeur du mot 8 bits}
{SPORT ENABLE}
IFC = 0x1E;                             {pas d'interruption SPORT }
ICNTL = 0;                              {pas interrupt nesting }
AX0 = 0x1C1F;                           {les deux ports série SPORTs actif, BWAIT et}
DM(0x3FFF) = AX0;                       {PWAIT en défaut}
IMASK = 0x1E;                           {interruption des ports séries actives}
            {—— END INITIALIZATIONS DES SPORT——}
```

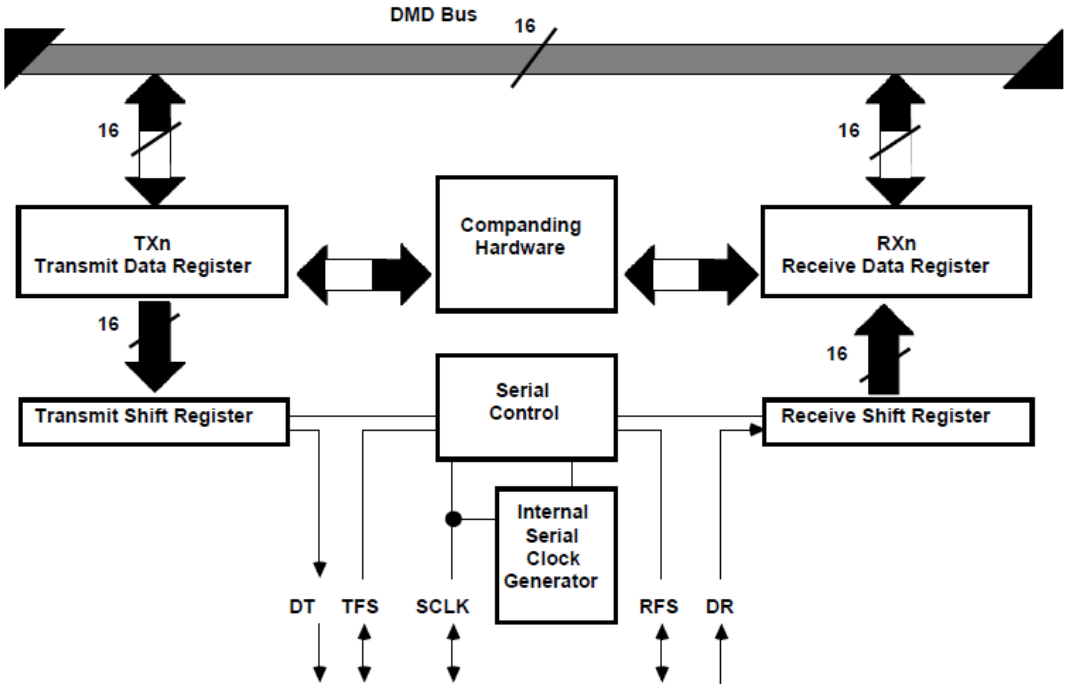


Fig. 32 Schéma block du port série.

4 Langage assembleur de la famille ADSP21xx.

4.1Architecture de la famille ADSP2100.

La figure 32 montre l'architecture du processeur ADSP2100 caractérisée par son architecture Harvard modifié (deux bus seulement à l'extérieur).

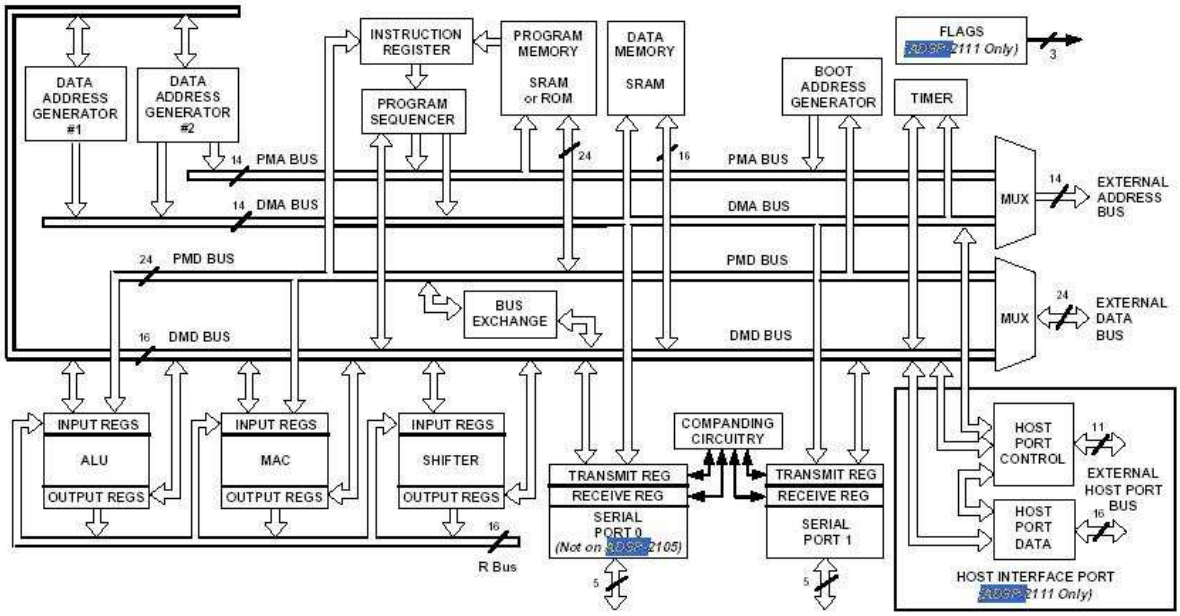


Fig.33 Architecture interne de l'ADSP 2100.

Il possède un bus R de 16 bit relié aux unités logiques (MAC, ALU et shifter) cette particularité permet l'utilisation du résultat d'une opération arithmétique d'une unité donnée comme entrée d'une autre unité, ceci lors d'une nouvelle opération. Il dispose d'un BUS EXCHANGE qui permet l'échange de données entre la mémoire programme donnée et la mémoire donnée.

Le MAC accepte des entrées sur 16x 16 bits et fournit un produit sur 32 bits en un seul cycle horloge. L'ADSP2100 est parmi les rares DSP a ne pas possédé des mémoires internes par contre il possède deux espaces mémoires externe un pour les données et l'autre pour les programmes. Sa mémoire cache lui permet de stocker jusqu'à 16 instructions. L'ADSP2100 intègre aussi l'adressage en mode bit inverse et des cycles d'arrêt par hardware.

Le constructeur ANALOG DEVICE propose aussi des DSP à virgule flottante tel que la famille ADSP21xxx. Les multiplieurs de cette famille acceptent des entrées sur 32 bits en virgule flottante et donne un produit sur 32 bits en virgule flottante. Il est équipé de deux accumulateurs virgule fixe de 80 bits. Le processeur possède 32x 48 bits instructions caches, une mémoire donnée extensible jusqu' a 4Gx40 bits mots et une mémoire programme extensible jusqu'à 16Mx 48bits mots en externe.

4.1.1 Exemple d'une instruction type ALU/MAC.

Le langage assembleur de la famille ADSP21xx est un langage semi évolué facile à lire et à écrire chaque code assembleur correspond à une instruction sur 24 bits exécutables en un cycle horloge. Ci-dessous une opération du type ALU avec condition

Conditional ALU/MAC operation, Instruction Type 9:

23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	Z	AMF					Yop		Xop			0	0	0	0	COND			

Conditional ALU/MAC operation, Instruction Type 9:

23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	0	0	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	1

C'est une multiplication du type X*Y

Avec Y= Y0 et X = AR

Avec la condition Not equal

Ci-dessous les codes des instructions utilisées :

AMF ALU / MAC Function codes

0 0 0 0 0	No operation	

MAC Function codes

0 0 0 0 1	X * Y	(RND)	
0 0 0 1 0	MR + X * Y	(RND)	
0 0 0 1 1	MR – X * Y	(RND)	
0 0 1 0 0	X * Y	(SS)	Clear when y = 0

ALU Function codes

1 0 0 0 0	Y	Clear when y = 0
1 0 0 0 1	Y + 1	PASS 1 when y = 0
1 0 0 1 0	X + Y + C	
1 0 0 1 1	X + Y	X when y = 0
1 0 1 0 0	NOT Y	

COND Status Condition codes

0 0 0 0	Equal	EQ
0 0 0 1	Not equal	NE
0 0 1 0	Greater than	GT
0 0 1 1	Less than or equal	LE
0 1 0 0	Less than	LT

X X Operand codes

0 0 0	X0 (SI for shifter)
0 0 1	X1 (invalid for shifter)
0 1 0	AR
0 1 1	MR0
1 0 0	MR1
1 0 1	MR2
1 1 0	SR0
1 1 1	SR1

Y Y Operand codes

0 0	Y0
0 1	Y1
1 0	F (feedback register)
1 1	zero

Z ALU/MAC Result Register codes

0	Result register
1	Feedback register

5. Implantation d'algorithmes de traitement numérique du signal sur DSP.

5.1Filtre FIR.

Dans ce qui suit nous aborderons le problème de filtrage numérique et de son implantation dans les processeurs numériques des signaux.

Soit l'équation aux différences suivante.

$$y(k) = \sum_{m=0}^{M} b_m . x(k-m) \qquad (1)$$

Dans ce système le signal de sortie dépend de la valeur présente et des valeurs précédentes du signal d'entrée.

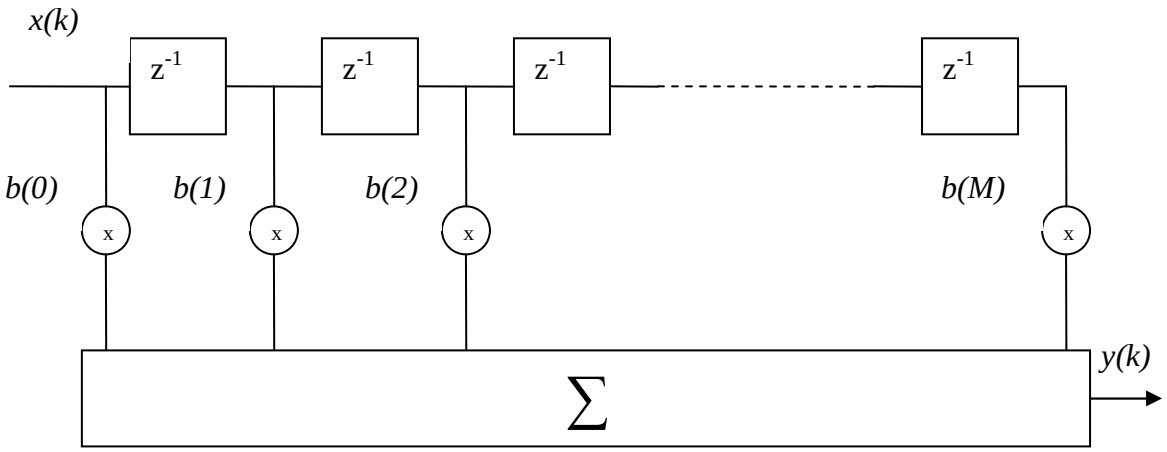


Fig.34 schéma bloc d'une réalisation récursive.

5.1.1. Implantation du filtre FIR.

Ci dessous les programmes d'un filtre a réponse impulsionnelle finie RIF (FIR), dans le cas du DSP d'Analog Device ADSP 2101 la configuration des registres est faite tel que :

- Entrée sortie sur le port serie0.
- L'horloge série est configurée d'une manière interne.
- L'horloge du processeur de 12.288 MHz est réduite à 1.536 MHz.
- L'horloge série est réduite a 8 KHz (frame sampling rate).

Ci-dessous le programme de configuration, le programme principal, la déclaration des constantes et la routine d'interruption.

Fichier du Setup et du programme principal.

```
.MODULE /RAM/ABS=0  routine principale;        {Le programme est charge de l'EPROM
MNAP=0}
.INCLUDE            <const.h>                  {Déclarations des constantes dans un autre fichier}
.VAR/DM /RAM/ABS=H#3800/CIRC      data_buffer [taps];     {Buffer circulaire Dans DM}
.VAR/PM /RAM/CIRC                 coefficient [taps] ;    { Buffer circulaire  Dans PM}
.Global                           data buffer, coefficient;
.External           fir_start;
.INIT               coefficient:< coefficient .dat>;  {Coefficients charger a partir d'un fichier externe}
        {Chargement des addresses du vecteur d'interruption}
                    JUMP restarter ; nop ; nop ; nop ;        {interruption aller a vecteur reset}
                    RTI; nop ; nop ; nop ;                    {interruption IRQ2 échantionnage}
```

```
                    RTI; nop ; nop ; nop ;              {interruption SPORT0 Transmit}
                    JUMP fir_start; nop; nop; nop;      {interruption SPORT0 Receive aller à
fir_start}
                    RTI; nop ; nop ; nop ;              {interruption SPORT1 Transmit}
                    RTI; nop ; nop ; nop ;              {interruption SPORT1Receivet}
                    RTI; nop ; nop ; nop ;              {interruption du Trimer}
```

{Initialisation}

```
Restarter :
                    L0 = %data_buffer;          {L0 est égal a  la longueur  «data_buffer»  et est
Un  buffer circulaire}
L4 = %coefficient;          { L4  est égal a  la longueur  «coefficient »  et est
Un  buffer circulaire}
M0=1; M4=1;                 { 1 pour des pas d'incrémentation égal a 1}
                    I0=^data_buffer;            {I0  pointe au début du buffer ou la donnée
                                                    commence}
                    I4=^coefficient;            {point ou les coefficients commencent}
                    CNTR =  %data_buffer;       {initialisation du compteur}
                    Do clear until CE;          {buffer initialiser avec des 0}
Clear:              DM(I0,M0) = 0;
                    I1 = H#3FEF;                {point de départ pour la phase de setup des
                                                différents registres de l'adresse H#3FF0 jusqu'a
                                                H#3FFF incrémentation par pas de 1 automatique
                                                après chaque lecture de la donnée}
DM (I1, M0) = H#0000; {H#3FEF: SPORT 1 AUTOBUFF Disabled}
                    DM (I1, M0) = H#0000; {H#3FF0 : SPORT 1 RSFDIV pas utilisée}
                    DM (I1, M0) = H#0000; {H#3FF1 : SPORT 1 SCLKDIV pas utilisée}
                    DM (I1, M0) = H#0000; {H#3FF2 : SPORT 1 CNTL pas utilisée}
                    DM (I1, M0) = H#0000; {H#3FF3 : SPORT 1 RSFDIV pas utilisée}
                    DM (I1, M0) = 191 ;     {H#3FF4 :8 KHz pour RFS}
                    DM (I1, M0) = H#0003; {H#3FF5: horloge série interne CLK a 1.536MHz}
                    DM (I1, M0) = H#6927;       {H#3FF6: Multichannel disabled}
                                                {Int. Gen serial clock}
                                                {Receive frame sync required}
                                                {Receive width 0}
                                                {Transmit frame sync required}
```

```
                                        {Transmit width 0}
                                        {Int Transmit frame sync disabled}
                                        {Int Receive frame sync enabled}
                                        {U-law companding}
                                        {8 bits word}
                DM (I1, M0) = 0000; {H#3FF7: Transmit Multichannel}
                DM (I1, M0) = 0000;
                DM (I1, M0) = 0000; {H#3FF9: Receive Multichannel}
                DM (I1, M0) = 0000;
                DM (I1, M0) = 0000; {H#3FFB: Timer not used, cleared}
                DM (I1, M0) = 0000;
                DM (I1, M0) = 0000;
                DM (I1, M0) = H#7000; { H#3FFE :DM Wait states}
                                        {H#3400 – H#37FF   D WAIT4}
                DM (I1, M0) = H#1000; {H#3FFF: SPORT0 enabled
                                        Boot page 0, 0 PM Waits, 0 boot Waits}
                ICNTL = H#00;
                IMASK = H#0018;   {enable SPORT0 interrupt only}
Mainloop:       IDLE;             {attendre une interruption}
                Jump Mainloop;
.ENDMOD;
```

Fichier initialisation des constantes

```
.CONST          taps =15; taps_less_one =14;
```

Routine d'interruption

```
.INCLUDE        <const.h>
.Entry          fir_start;                  {étiquette visible en dehors du module}
.External       data_buffer, coefficient;   {variable globale visible dans le  module}
FIR_START:      CNTR = taps_less_one;       {compteur = N-1}
                SI = RX0;                   {lecture du SPORT0}
                DM (I0, M0) = SI;           {chargement du buffer données}
                MR = 0, MY0 = PM (I4, M4), MX0 = DM (I0, M0);
                                            {Chargement de la donnée et du coeff
Avec incrémentation  automatique par pas de 1 pour la multiplication à venir}
                Do convolution until CE
```

Convolution: MR = MR+ MX0* MY0 (SS), MY0 = PM (I4, M4), MX0 = DM (I0, M0);

MR = MR+ MX0* MY0 (RND),

TX0 = MR1; *{écriture dans le SPORT}*

RTI;

{Return from interrupt}

.Endmod;

5.2. Systèmes à réponse impulsionnelle infinie (RII ou IIR).

Les filtres RII sont des réalisations récursives caractérisées par des réponses impulsionnelle de durée infinie, avec des échantillons g(k) non nuls sur l'intervalle L tel que $k_0 \leq k \leq +\infty$.

L'opération de filtrage est caractérisée dans ce cas par l'équation aux différences suivante :

$$y(k) = \sum_{m=0}^{M} \frac{b_m}{a_0}.x(k-m) - \sum_{n=1}^{N} \frac{a_n}{a_0}.y(k-n) \qquad (2)$$

Dans ce système le signal de sortie dépend du signal d'entrée et des valeurs précédentes du signal de sortie.

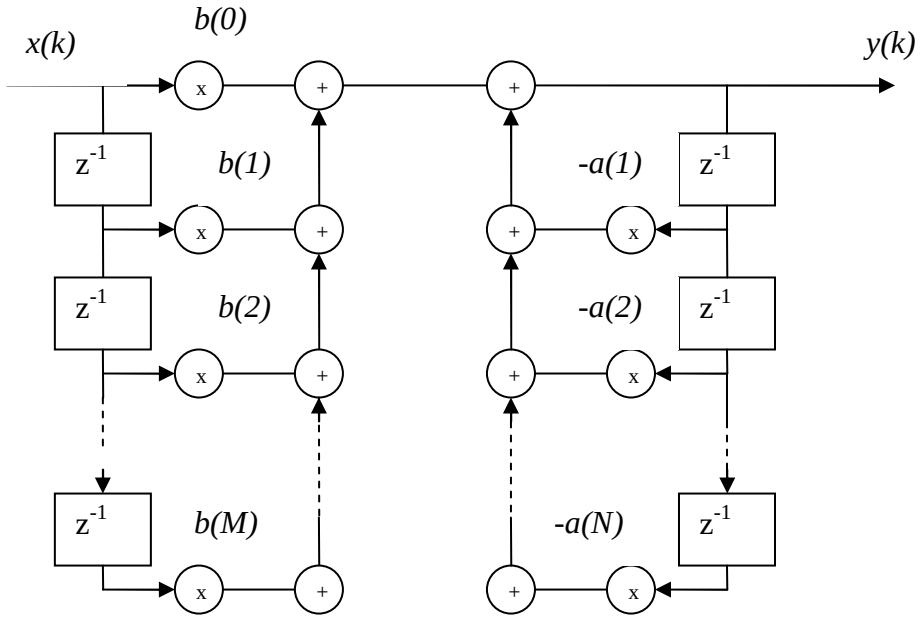


Fig.35 schéma bloc d'une réalisation non récursive.

Nous pouvons considérer la fonction de transfert G(z) correspondant à l'équation (2) comme la mise en série de deux systèmes.

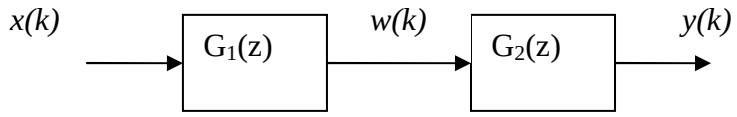


Fig. 36 Schéma bloc global .

$$G_1(z) = \frac{W(z)}{X(z)} = \frac{1}{\sum_{n=0}^{N} a_n . z^{-n}} \tag{3}$$

$$x(k) = \sum_{m=0}^{M} a_n . w(k-n) \tag{4}$$

$$G_2(z) = \frac{Y(z)}{W(z)} = \sum_{n=0}^{M} b_m . z^{-m} \tag{5}$$

$$y(k) = \sum_{m=0}^{M} b_m . w(k-m) \tag{6}$$

5.2.1. Implémentation du filtre IIR.

Dans ce qui suit nous proposons une routine pour l'implémentation du filtre IIR dans le cas d'un ADSP21xx.

```
.MODULE diriir_sub;
{Direct Form IIR Filter Subroutine
    Calling Parameters
        MR1 = Input sample (x[n])
        MR0 = 0
        I0  →   Delay line buffer current location (x [n-1])
        L0 = Filter length
        I4  →   Delay line buffer current location (y [n-1])
        L4
        I5  →   Feedback coefficients (a's)
        L5 = Filter length - 1
        I6  →   Feedforward coefficients (b's)
        L6 = Filter length
```

```
        M0 = 0
        M1, M4 = 1
        M2 = 2
        CNTR = Filter length - 2
        AX0 = Filter length – 1
    Return Values
        MR1 = output sample (y[n])
        I0  →   delay line current location (x [n-1])
        I5  →   feedback coefficients les a (i)
        I6  →   feedforward coefficients les b (i)
        I4  →   delay line current location (y [n-1])
    Altered Registers
        MX0,MY0,MR
    Computation Time
        (N - 2) + (N - 1) + 10 + 4 cycles (N = M = Filter order)
    All coefficients and data values are assumed to be in 1.15 format.  }
.ENTRY  diriir;
        CNTR = Filter length - 2
Diriir: MX0=DM (I4, M1), MY0=PM (I5, M4); {chargement de la donnée et du coeff
Avec incrémentation  automatique par pas de 1 pour la multiplication à venir}
    DO poleloop UNTIL CE;       {boucle de calcul du filtre pour les a(i)}
Poleloop:       MR=MR+MX0*MY0 (SS), MX0=DM (I4, M1), MY0=PM (I5, M4);
    MR=MR+MX0*MY0 (RND);
    CNTR=AX0;                   {initialisation du compteur a N-1 pour les b(i)}
    DM(I4,M0)=MR1;              {écriture des y[n-1] dans le buffer circulaire pas égal a 0}
    MR=0, MX0=DM (I0, M1), MY0=PM (I6, M4);
    DO zeroloop UNTIL CE;       {boucle de calcul du filtre pour les b(i)}
Zeroloop:       MR=MR+MX0*MY0 (SS), MX0=DM (I0, M1), MY0=PM (I6, M4);
    MR=MR+MX0*MY0 (RND);
      MODIFY (I4,M2);           {pas du buffer =M2+position de I4 pour buffer des y[n-1]}
    RTS;
.ENDMOD;
```

ADSP-2100 Family Benchmarks For Digital Control Applications		
	ADSP-2101 (60 ns)	ADSP-2105 (100 ns)
PID Loop	0.48 µs	0.8 µs
FIR Filter	60 ns/tap	100 ns/tap
IIR Biquad – 16 Bit (2nd Order)	0.72 µs	1.2µs
IIR Biquad – 16 Bit (nth Order)	4N + 4 cycles	4N + 9 cycles
IIR Biquad – 32 Bit (2nd Order)	2.64 µs	4.4 µs
IIR Biquad – 32 Bit (nth Order)	16N + 12 cycles	16N + 22 cycles
256-Point FFT (Complex)	0.405 ms	0.675 ms
Matrix Multiply (3x3 * 3x1)	3.12 µs	5.2 µs
Stochastic Gradient (LMS) N Tap Coefficient Update	2N + 9 cycles	2N + 9 cycles

Tableau1 Exemple de temps de cycle horloge

5.3. L'algorithme LMS.

Un système adaptatif est un système où les coefficients s'ajustent tous seuls suite à une variation quelconque du signal d'entrée. Les filtres adaptatifs sont utilises dans le cas d'une connaissance incertaine des caractéristiques du signal, ou dans le cas d'une variation de ces dernières.

5.3.1. Structure de L'algorithme LMS.

La figure 37 montre la structure d'un filtre adaptatif par soustraction de bruit. Nous étudierons le cas du filtre à réponse impulsionnelle finie (R.I.F).

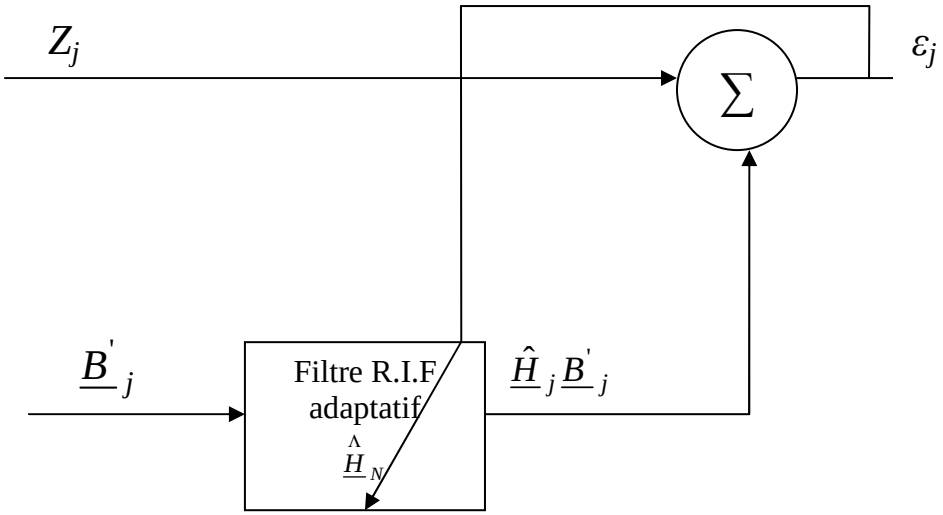


Fig.37 structure d'un filtre adaptatif soustracteur de bruit.

Le signal d'entrée est perturbé par un bruit B(t). Pour éliminer ce bruit nous avons besoin a l'entrée du filtre adaptatif d'un bruit B'(t) corrélé avec B(t). Une fois le système adapté nous obtenons une estimation $\hat{B}(t)$ du bruit à éliminer.

L'algorithme utilisé est l'algorithme LMS (Least Mean Square), ou MCM (Moindre Carrés Moyens) de Widrow. Les coefficients sont calculés continuellement suivant l'équation (2).

B'(t): référence bruit.

$\underline{\hat{H}}$: Coefficients du filtre adaptatif. (1)

μ: adaptive gain where:

$$\underline{\hat{H}}_{j+1} = \underline{\hat{H}}_j + 2\mu\varepsilon\underline{B'}_j \quad (2)$$

$$\varepsilon_j = Z_{j-}\underline{\hat{H}}_j^T\underline{B'}_j \quad (3)$$

$$\underline{\hat{H}}_j^T = \left[\hat{H}_1, \hat{H}_2, ...\hat{H}_M\right] \quad (4)$$

Z_j : Signal original *Xj*+ *Bj*.

Xj : Signal a estimé.

Bj : Bruit.

ε_j : Estimation du signal.

5.3.2. Implantation de L algorithme LMS.

Pour pouvoir implanter l'algorithme LMS sur le DSP il faut déjà l'arranger de manière à ce qu'il soit utilisable pour le calcul numérique. Les équations de l'erreur et des coefficients eq(8 et 9) deviennent :

$$\hat{H}_k(n+1) = \hat{H}_k(n) + 2\mu\varepsilon(n).B'(n-k+1) \quad (5)$$

$$\varepsilon\ (n) = Z\ (n) - \sum_{k=1}^{n}\hat{H}_k(n).B'(n-k+1) \quad (6)$$

Le listing du programme suivant comporte l'algorithme LMS et la configuration des registres systèmes, les registres d'interruptions, les buffers circulaires…...et ceci pour le processeur ADSP2105 d'ANALOG DEVICE.

```
.MODULE/RAM/ABS=0   main;
.CONST              ND = 85;
 .VAR/DM/RAM/ABS=H#1FFD     beta;
 .VAR/DM/RAM/CIRC          data_adapt1 [32];   {fir voie1 adapt-data delay line}
 .VAR/PM/RAM/CIRC          coefficient1[32];  {fir voie1-adapt}
```

```
            JUMP restart; NOP; NOP; NOP;
            JUMP FIR_PB_conv; NOP; NOP; NOP;
            RTI;NOP;NOP;NOP;
            RTI;NOP;NOP;NOP;
            RTI;NOP;NOP;NOP;
            RTI;NOP;NOP;NOP;
            RTI; NOP; NOP; NOP;
Restart:        AY0=H#0800;        {system command}
            DM(H#3FFF)=AY0;
            ay0=h#1000;
            dm(H#3FFE)=ay0;     {dm wait}
            ay0=4881;     {9761 pour l'échantillonnage a 512hz sclkdiv}
            Dm (h#3FF1)=ay0;
            ay0=h#4000;        {clk ext}
            Dm (h#3FF2) =ay0;
            I2=^data_adapt1;   {données FIR ADAPT}
            I6=^coefficient1;    {coeff FIR ADAPT   }
            L2=32; L6=32;
            M1=1; M6=2; M3=-1;     M7=-1; M0=1;   M4=1; M2=0; M5=0;
            ax0=dm(h#3400);    {initialisation CAN}
            {Enable irq2-}
            ICNTL=H#14;
            ax0=h#20;          {IRQ2 validée relier au CAN par exemple}
            Mask=ax0;
Wait:           NOP;
            JUMP wait;
             ay0=dm(h#3400);
            CNTR=31;
            DM (I2, M1)=ay0;              {charger les données}
            Mr=0, my0=pm (I6, M4), mx0=dm (I2, M1);    {calcul de l'estimation du bruit  n(t)
                                                        Convolution avec les coefficients du filtre
            DO conv1 UNTIL CE;                          adaptatif}
conv1:          mr=mr+mx0*my0 (ss), my0=pm (I6, M4), mx0=dm (I2, M1);
Mr=mr+mx0*my0 (RND);
```

```
            IF MV SAT MR;
                                    { (ay0 : signal +bruit} ; ar : bruit estimé  après
                                     Adaptation avec le filtre adaptatif}
            Ar=-mr1;
            Ar=ar+ay0;              {epsilon(t)}
          {   DM(H#3400)=ar; }      {sortie voie1}
            Dm (z1)=ar;
            mx0=ar;
            my1=dm(h#1FFD);         {  mémoire ou est stockée le bêta du filtre adaptatif }
            CNTR=32;                                          {longueur du filtre adaptatif
            mf=mx0*my1(RND), mx0=DM (I2, M1);          et procédure de calcul des
coefficients
            mr=mx0*mf(RND),  ay0=PM (I6, M4);      avec stockage dans le buffer
                                                        des   coefficients}
 DO adapt1 UNTIL CE;
            ar=mr1+ay0, mx0=DM (I2, M1), ay0=pm (I6, M7);
 adapt1:        PM (I6, M6) =ar, mr=mx0*mf (RND);

            MODIFY(I2,M3);                    { synchronisation des buffers des données }
            MODIFY(I6,M7);                    { initialisation des buffers des coefficients }
RTI ;
.ENDMOD ;
```

Chapitre 3: Les microcontrôleurs

1. Introduction.

Un microcontrôleur est un circuit intégré comprenant essentiellement une unité centrale qui est le cœur du système appelé également CPU (Central Processing Unit), une mémoire vive RAM (Ramdom Access Mémory) une mémoire morte ou ROM, EPROM (Read Only Memory), des ports d'entrée sorties, et des périphériques (Timer, des convertisseurs :CAN/CNA, des interfaces séries…..) des bus pour la communication. Un microcontrôleur est donc plus adapté aux applications embarquées. On le trouve dans plusieurs domaines telque les télécommunications (téléphone, télécopieur, réseaux…), l'industrie (automate programmable, régulation de niveau..) l'automobile (auto radio, calculateur, …) … L'intégration de tout ces éléments dans un seul boitier donne un composant autonome qu'on peut intégrer facilement dans une application à condition de choisir le microcontrôleur approprié.

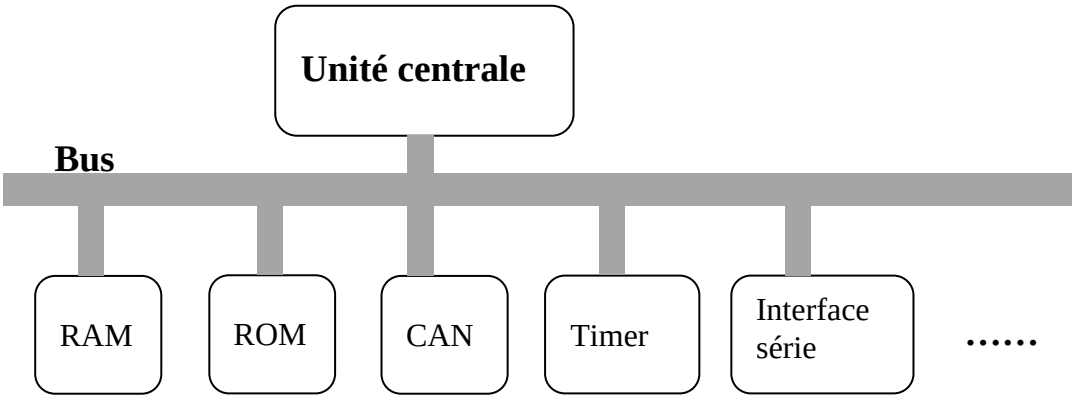


Fig.38 Schéma fonctionnel d'un microcontrôleur.

2. Les microcontrôleurs de MOTOROLA.

Il existe plusieurs constructeurs de microcontrôleurs (siemens, Texas instrument intel, Motorola,….). Nous nous intéresserons dans ce cours à un des constructeurs les plus connu sur le marché : Motorola. Et plus particulièrement au microcontrôleur 68332 de la famille 683xx .

La famille 6805 est la famille la plus ancienne de Motorola. Un microcontrôleur type de cette famille comprend typiquement un cœur (CPU sur 8 bits), une ROM, une RAM, un Timer des bus et trois port d'entrée sortie dont un analogique. La figure 39 montre l'évolution de la famille de MOTOROLA

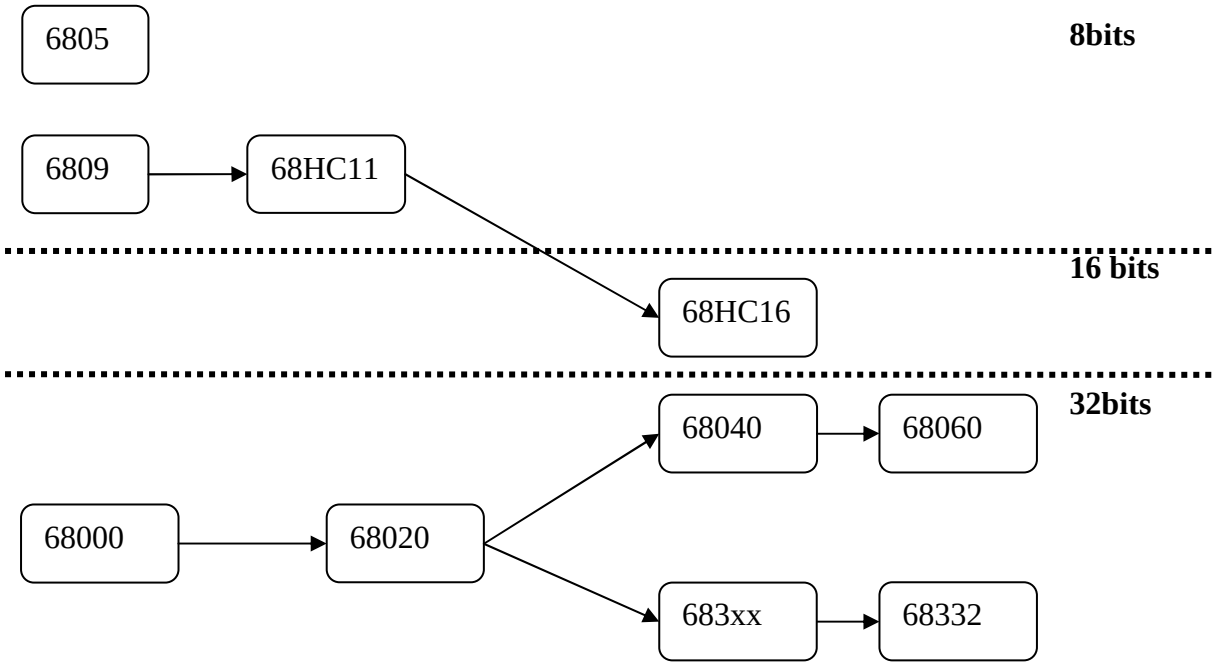


Fig.39 Evolution des micro controleur de la famille motorola.

A part la famille 6805 qui possède un CPU propre à elle les autres versions plus récentes de microcontrôleurs de Motorola possède des CPU issue de microprocesseur classique telque le 6809 pour les microcontrôleurs HC11 et HC16 et les processeurs 68000 et 68020 pour la famille de microcontrôleurs 683xx, comme le montre la figure 39.

3 Description du microcontrôleur 68332.

Le microcontrôleur 68332 de Motorola est un microcontrôleur 32 bits il est construit autour de modules standard interfacé par un bus interne (IMB).

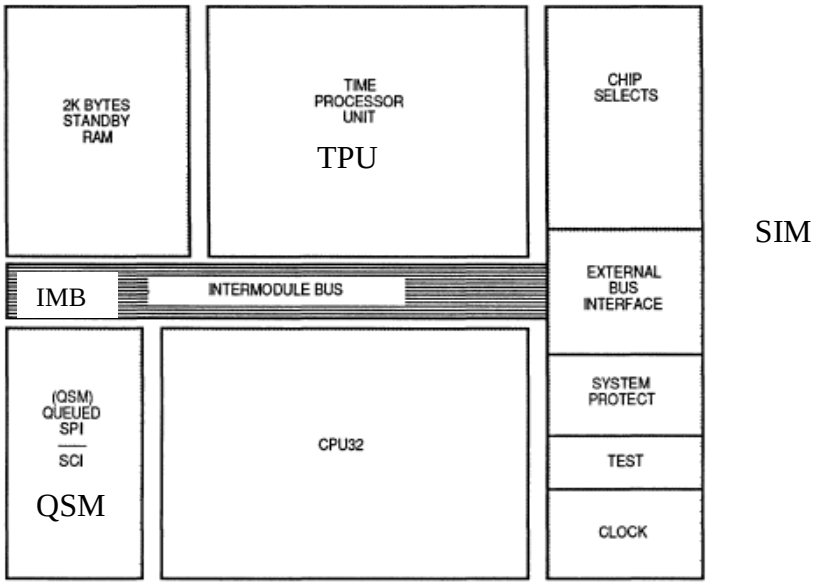


Fig.40 Schéma block du MC68332.

Il est donc constitué d'un processeur 32 bits (CPU32), d'un module d'intégration (SIM), d'un module de gestion de temps (TPU), d'un module interface série (QSM) d'une RAM statique de 2k byte.

3.1 Constituant du microcontrôleur 68332.

3.1.1 Le module processeur CPU32.

Le modèle de programmation du CPU32 est formé de deux groupes de registres un groupe pour le mode utilisateur et un groupe pour le mode superviseur, ce dernier permet à l'utilisateur de protéger certaines fonctions systèmes.

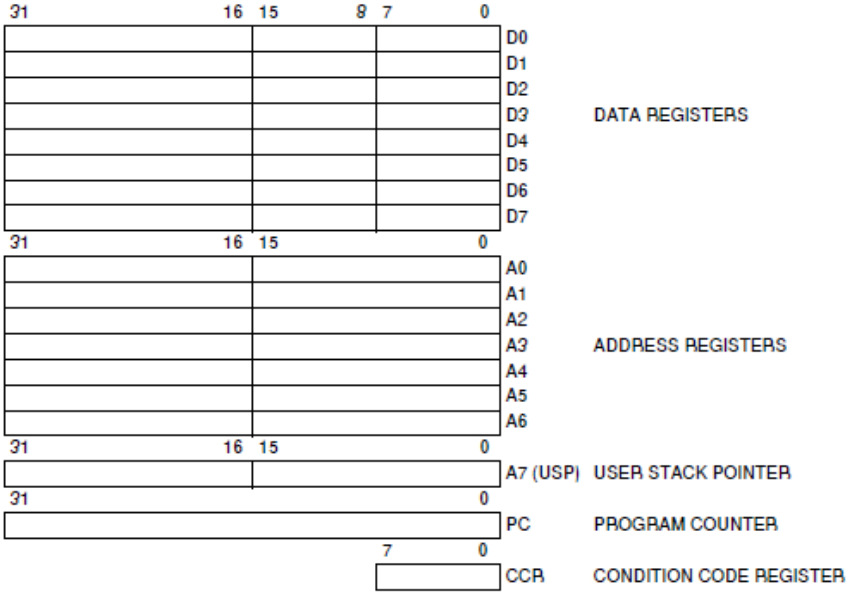


Fig.41 Registre du modèle de programmation utilisateur.

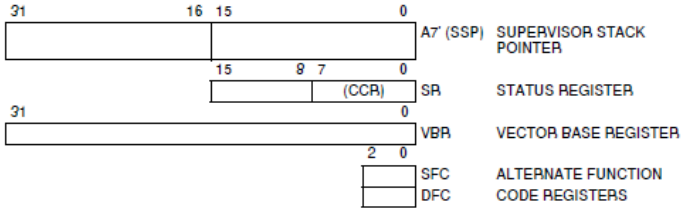


Fig.42 Registre du modèle de programmation superviseur.

Comme le montre les figures 41 et 42 le CPU 32 possède 8 registres données sur 32 bits 7 registres adresses sur 32 bits un pointeur de programme sur 32 bits, des pointeurs de piles sur 32bits pour le mode superviseur et pour le mode utilisateur, un registre de base vecteur VBR sur 32 bits et un registre d'état (STATUS) sur 16 bits. Les registres donnés D0 à D7 supporte

des données de type bits, Byte ou octet (8bits), Word ou mot (16 bits), Long Word ou mot long (32 bits) et Quad Word (64 bits).

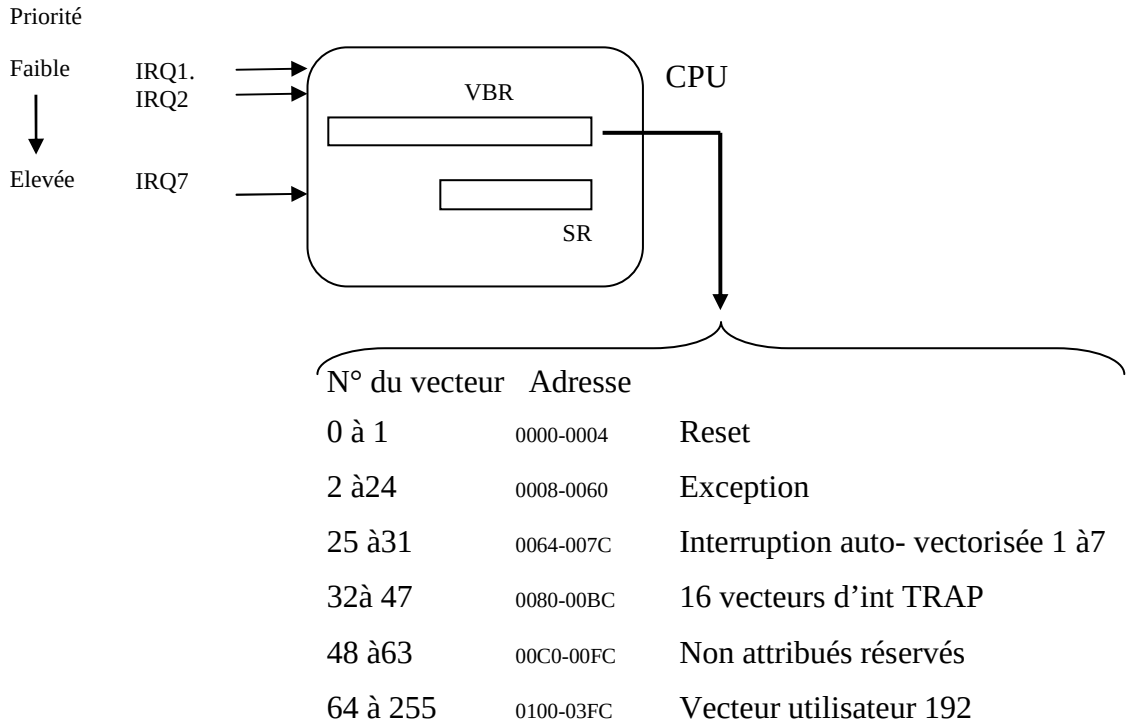


Fig 43. Table des vecteurs d'interruption.

Les interruptions des différents modules sont réceptionnées sur les lignes IRQ 1à7 comme le montre la figure 43**.** Le CPU ne prendra en compte cette interruption qu'après avoir exécuté l'instruction présente d'une part et avec une priorité supérieure au masque d'interruption du registre SR. Les adresses des vecteurs d'interruption sont données à la figure 43.

3.1.1.1 Les Instructions du CPU.

Les instructions du CPU 32 sont similaires aux instructions du processeur 68020 de Motorola

-Format d'une instruction assembleur

Le format d'une instruction écrite en assembleur est la suivante :

Étiquette : ***Mnémonique.format*** ***source***, ***destination*** ***; commentaires***

Étiquette : elle permet d'accéder à une instruction.

Mnémonique : l'instruction à réaliser en général c'est un diminutif de l'action à faire.

Format : format des données (taille).

. B : pour une donnée de type octet (byte) 8 bits

.W pour une donnée de type mot word 16 bits

.L pour une donnée de type mot long long word 32 bits

Source : départ

Destination : arrivée

Ci- dessous quelque exemple d'instruction du processeur 68000 de Motorola

Instructions de transfert :

-Transfert simple :

MOVE.*(size)* *<ea>, <ea->*

Size: *.(B,W,L)*

Transfert de la donnée de la source vers la destination

Exemple :

Adressage immédiat

MOVE.W #142,D0 transféré 142 vers le mot de poids faible de D0

Adressage absolue

MOVE.B #$A2, $FE8002 Écrit la donnée A2 à l'adresse FE8002.

Adressage indirect

MOVE.B D0, (A_2)

L'adresse de la destination est dans un pointeur d'adresses.

Soit :

D0= C54F3152

A_2 =00EE7200

Après exécution de l'instruction on trouvera 52 dans la cellule mémoire d'adresse 00EE7200

-Transfert vers registre d'adresse

MOVE.*(size)* *<ea>, <A_n->*

Size: *.(W,L)*

Transféré le contenu de la source au registre d'adresse de la destination

Exemple:

MOVEA.*L* *#$7000, A_2*

-Chargement d'une adresse

LEA. . *(Size)*

Transférer dans un registre d'adresse l'adresse de l'opérande

LEA.L (A2), A1

A2 vers A1

Size: *(L)*

-Instructions arithmétiques

Addition, Soustraction, Multiplication, Comparaison :

Exemples

-Addition

ADD. *(Size)* <Donnée>, Dn

Size: *(B, W, L)*

< Donnée > + Dn vers Dn sur un Byte

ADDI. (size) #valeur, <data->

Size: . *(B, W, L)*

Addition de la donnée immédiate avec la destination et mettre le tout dans la destination

-Soustraction

SUB. (Size) <data>,Dj

Soustraire la donnée (data) de Dj et mettre le résultat dans Dj.

SUBI.(size) #valeur,<data->

Size: *.(B,W,L)*

-Multiplication

MULS.(size) <data>,Di

Size: *.(W)*

Source *Destination dans destination

-Comparaison

CMP.(B,W,L) <data>,Dj

Soustraire la source de la destination et mettre le code condition (cc) avec la valeur corespondante.

Instructions Logiques

OR.(size) <data>,Dj

Size: *.(B,W,L)*

Faire une opération « ***ou*** » logique entre la donnée et la destination et enregistré le résultat dans la destination.

AND.(size) <data>,Dj

Size: *.(B,W,L)*

Faire une opération « ***et*** » logique entre la donnée et la destination et enregistré le résultat dans la destination.

Instructions de sauts

Elles sont divisées en deux catégories :

Instruction de saut inconditionnel JMP, BRA

Instruction de saut inconditionnel Bcc, (DBcc) cc : code condition

Les différentes conditions (***cc***):

CC retenue à 0 C	***Arithmétique signée***
CS retenue à 1 C	GE supérieur ou égal $N.V + N.V$
EQ égal Z	GT supérieur N.V.Z + N.V.Z
NE non égal Z	LE inférieur ou égal Z + N.V + N.V
MI négatif N	LT inférieur N.V + N.V
PL positif N	***Arithmétique non signée***
VC pas de dépassement V	HI plus grand C.Z
VS dépassement V	LS plus petit ou égal C+ Z

Exemple

TOTO	ADD.W	A2,D3	Additionner le registre source A2 avec le registre Destination D3 et mettre le résultat dans le registre Destination D3.
	BRA	TOTO	saut a l'étiquette TOTO sans condition

3.1.2 Le module d'intégration SIM

Le module d'intégration système figure 44 est formé par les cinq sous modules suivant, configuration du système et protection, génération de l'horloge système, sélection de périphérique, l'interface bus externe et test du système.

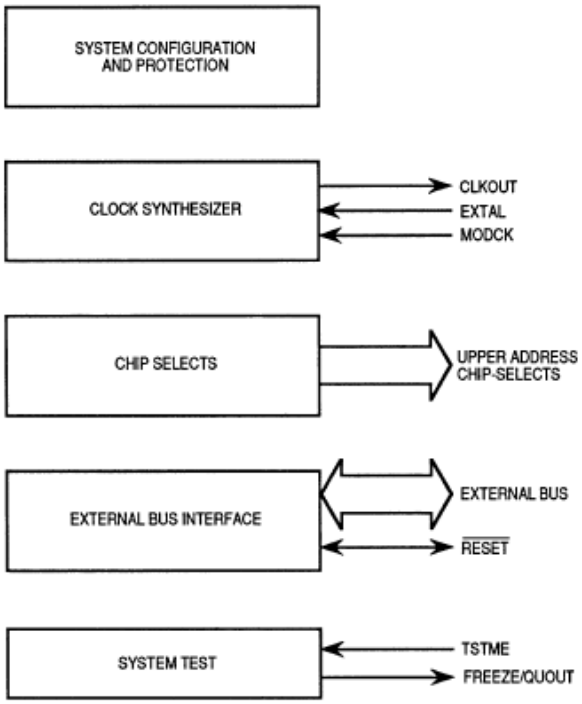


Fig.44 Schéma bloc du module intégration (SIM).

- Le sous module configuration du système et protection (SYSTEM CONFIGURATION AND PROTECTION) est destiné à la configuration des paramètres du microcontrôleur et a la génération de chien de garde (watchdog)
- Le sous module génération de l'horloge système (CLOCK SYNTHESIZER) permet de générer des signaux horloges qui sont utiliser par le module SIM, d'autre module ou même des composants externes.
- Le sous module sélection des périphériques (CHIP SELECTS) peut fournir 12 signaux de sélection (chip select) a chaque signal est associé un registre de base et un registre option qui contiennent les caractéristiques du signal générer.
- Le sous module interface bus externe (EXTERNAL BUS INTERFACE) il permet le transfert des informations entre le CPU la mémoire les périphériques…
- Le sous module test du système (FACTORY TEST) comporte tout le hardware nécessaire pour le test du microcontrôleur il est utilisé par le fabricant pour faire des tests usines.

3.1.3 Le module interface série (QSM).

Le module interface série offre au microcontrôleur deux interface de communication une interface de communication série synchrone (full-duplex) la QSPI et une interface série asynchrone (full-duplex) la SCI ces deux modules fonctionnent indépendamment l'un de l'autre.

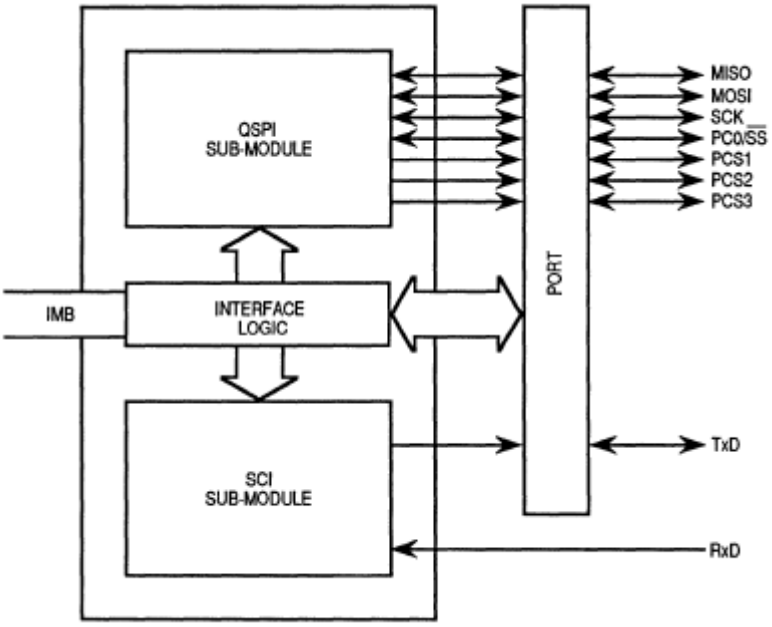


Fig.45 Schéma bloc du module QSM.

L'espace donnée superviseur est constitué de registres pour l'intégration du module QSM dans le microcontrôleur MCU l'accès a ces registres n'est permis que si le processeur est en mode superviseur.

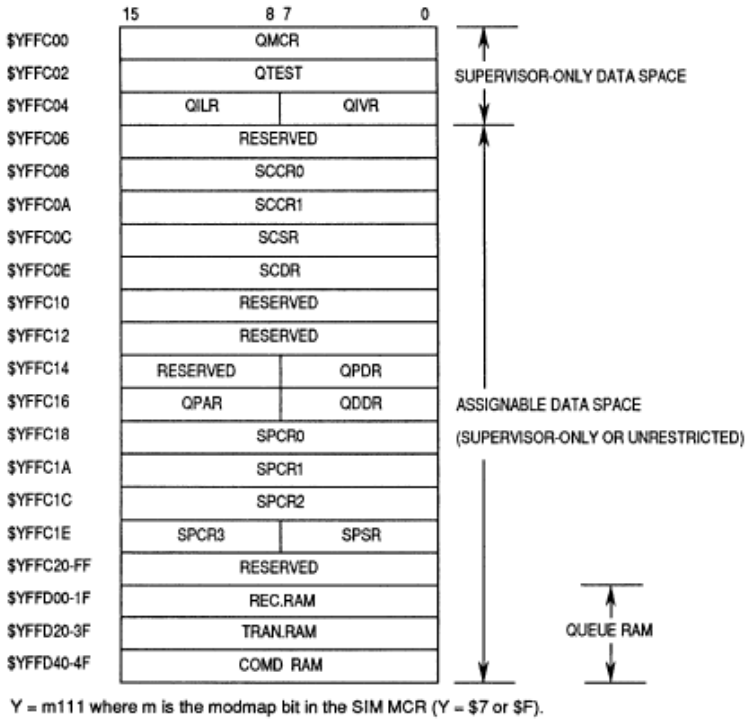


Fig.46 Différentes adresses des registres du module QSM.

La figure 46 montre l'organisation mémoire des registres du module QSM.

3.1.4 Le module mémoire RAM.

Le module mémoire est une RAM de 2048 bits elle peut être utilisé par le processeur comme des piles de mémoire ou comme une zone de stockage de variables. Elle peut aussi être utilisée par le module TPU.

3.1.5 Le module TPU.

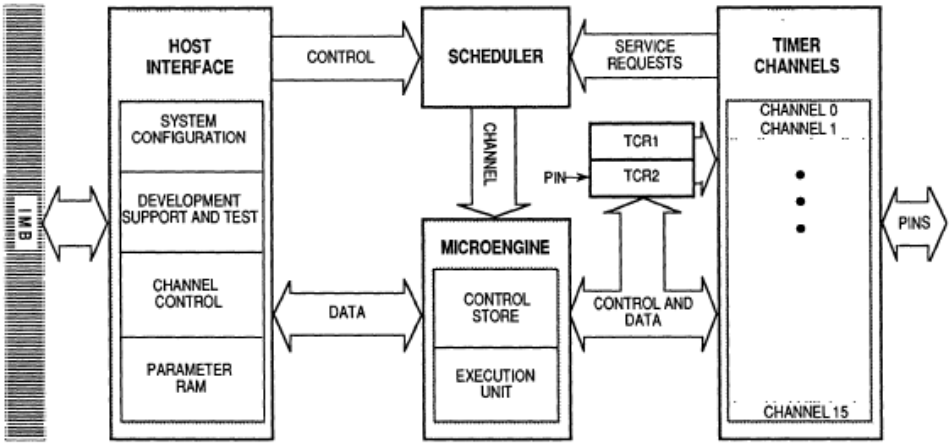


Fig.47. Schéma bloc simplifié du module TPU

Le module TPU communique avec le reste des constituants du microcontrôleur à l'aide du BUS inter-module IMB il est formé des cinq unités principales suivantes. L'unité Host interface, 16 canaux pour la gestion du temps, l'unité scheduler, l'unité microengine et l'unité d'exécution.

3.1.5.1 Adressage du registre TPU.

Les registres du module TPU sont classés en 4 grandes catégories :

- Registres de configuration système.
- Registres status et contrôle des canaux.
- Registre de développement support et test et vérification.
- Registres de paramétrage des canaux.

Tous ces registres sont des registres de 16 bits le tableau suivant donne l'organisation mémoire des registre du module de gestion de temps TPU.

	ADDRESS	15 BYTE n 8 | 7 BYTE n + 1 0 (WORD)
S	$YFFE00	MODULE CONFIGURATION REGISTER
S	$YFFE02	CONFIGURATION REGISTER
S	$YFFE04	DEVELOPMENT SUPPORT CONTROL REGISTER
S	$YFFE06	DEVELOPMENT SUPPORT STATUS REGISTER
S	$YFFE08	INTERRUPT CONFIGURATION REGISTER
S	$YFFE0A	INTERRUPT ENABLE REGISTER
S	$YFFE0C	CHANNEL FUNCTION SELECT REGISTER 0
S	$YFFE0E	CHANNEL FUNCTION SELECT REGISTER 1
S	$YFFE10	CHANNEL FUNCTION SELECT REGISTER 2
S	$YFFE12	CHANNEL FUNCTION SELECT REGISTER 3
X	$YFFE14	HOST SEQUENCE REGISTER 0
X	$YFFE16	HOST SEQUENCE REGISTER 1
X	$YFFE18	HOST SERVICE REQUEST REGISTER 0
X	$YFFE1A	HOST SERVICE REQUEST REGISTER 1
S	$YFFE1C	CHANNEL PRIORITY REGISTER 0
S	$YFFE1E	CHANNEL PRIORITY REGISTER 1
S	$YFFE20	INTERRUPT STATUS REGISTER
S	$YFFE22	LINK REGISTER
S	$YFFE24	SERVICE GRANT LATCH REGISTER
S	$YFFE26	DECODED CHANNEL NUMBER REGISTER

S = Supervisor accessible only
X = Assignable as supervisor accessible only (if SUPV = 1) or unrestricted (if SUPV = 0). Unrestricted allows both user and supervisor access.
Y = m111, where m is the state of the modmap bit in the module configuration register of the system integration module (Y = $7 or $F).

Fig.48 Différentes adresses des registres du module TPU.

3.1.5.2 Les fonctions de gestion de temps du module TPU.

Le module de gestion de temps TPU est muni de 16 canaux capables de réaliser une multitude de fonctions dépendantes du temps. Ces taches peuvent être simples ou complexes parmi les

fonctions les plus courantes réalisées sont : Entrée sortie logique, comparateur, modulation de largeur d'impulsion, modulation de largeur d'impulsion synchronisée, mesure de période, Moteur pas à pas….

Le tableau suivant montre les fonctions possibles avec le module TPU on accède à ces fonctions en configurant les registres *CFSR0, CFSR1, CFSR2, CFSR3* aves les codes de la fonction correspondante (colonne2) de ce tableau et les registres HSRR0 et HSSR1 avec les codes de la colonne 3 du tableau (*Host service Request code*) et enfin les registres HSQR0 et HSQR1 avec les codes de la colonne 4 du tableau (*Host Sequence Code*).

Nom de la fonction	Code de la fonction	Host service Request code	Host Sequence Code
PPWA	$F	0=None 1= non utilisé 2=initialisation 3= non utilisé	0 = 24 bit period 1 = 16 bit periode + link 2 = 24 bit pulse width 3 = 16 bit pulse width + link
OC Output Compare	$E	0 = none 1 = host initialed pulse mode 2 = non utilisé 3=continuous p ulse mode	0 = execute all functions 1 = execute all functions 2 = only update TCRn parameters 3 = only update TCRn parameters
SM Stepper motor	$D	0 = none 1= none 2= initialization 3 = step request	Non utilise
PSP Position-synchronized pulse generator	$C	0 = none 1 = immediate update request 2= initializatio 3= force change	0 = pulse width set by angle 1 = pulse width set by time 2 = pulse width set by angle 3= pulse width set by time
PMA/PMM Period measurement with additional missing transition detect	$B	0 = none 1 = initialization 2 = non utilise 3= non utilise	0 = PMA bank mode 1 = PMA count mode 2 = PMM bank mode 3= PMM count mode
ITC Input Capture/ input transition counter	$A	0 = none 1 = initialization 2 = non utilise 3= non utilise	0 = no link, single mode 1 = no link, continuous mode 2 = link, single mode 3 = link, continuous mode
PWM Pulse Width Modulation	$9	0 = none 1 = Immediate update request 2 = initialization 3 = non utilise	Non utilise
DIO Discrete Input/ouput	$8	0 = None 1 = Force ouput High 2 = force ouput Low 3 = initilization, input spcecified 3 = initialization, periodic input, 3= update pin status parameter	0 = trans mode = record pin on transition 1 = Match mode record pin at MATCH_RATE 2 = Record pin state on HSR11
SPWM Synchronized pulse width modulation	$7	0 = none 1 = non utlisé 2 = initialization 3 = Immediate update request	0 = mode 0 1 = mode 1 2 =mode 2 3 = non utilise
QDEC Quadrature decode	$6	0x = no action 10 = read TCR1 11 = Initialize	X0 = primary channel X1 = secondary channel

Tableau2 Les différents registres du module TPU.

3.1.5.3 Exemple de configuration des canaux en mode entrée/ sortie TOR.

Soit un programme qui mémorise les entrées du port B, allume une diode, éteint une autre après une temporisation enfin éteint toutes les led.et mémorise les entrées du port B.

```
/****************************************
* FONCTION PRINCIPALE
****************************************/
main()
{
/* INITIALISER
*******************************/
/* Dans ce qui suit nous configurons  le port A du micro contrôleur en mode "Discet Input
Output" (DIO)-pour cela on doit mettre la valeur  $8  sur 4 bits pour chaque canal du port A
de channel 15 a channel 0 voir Tableau2 */
CFSR3=0x8888;                          /* CHA0 à CHA3 en mode "DIO" */
CFSR2=0x8888;                          /* CHA4 à CHA7 en mode "DIO"*/
CFSR1=0x8888;                          /* CHA8 à CHA11 en mode "DIO" */
CFSR0=0x8888;                          /* CHA12 à CHA15 en mode "DIO" */
CPR1=0xFFFF;                           /* Tous les bits de PA en priorité haute */
CPR0=0xFFFF;                           /* Tous les bits de PB en priorité haute */
        move.w      #$ 5555,HSRR1      /* éteindre les led Tous les bits du port A à 1
                                       mettre dans  tous les canaux (CH0 à CH 7) du
                                       HSRR1 la valeur 1 (forcé les sorties a l'état haut)
        move.w      #$ AAAA,HSQR0      /* les entrées seront mémorisées
                                       Mettre dans  tous les canaux (CH8 à CH 15) du
                                       HSQR0 sur deux bits  la valeur 2 (mémorisé les
                                       entrées dans le registre HSR11)

/*************************
 BOUCLE PRINCIPALE
************* ************/
Prog_p  move.w      #$ 5556, HSRR1     /*Mettre dans les deux Bits du CH0 du HSRR1 la
                                        valeur 2 en binaire (forcé la sortie à  l'état bas)  et
                                       mettre dans tous les autre canaux (CH1 à CH 7) du
                                       HSRR1 la valeur 1 (forcé les sorties a l'état haut)
```

```
                                                                    */
        jsr         Tempo           /*appeler le sous programme de temporisation*/
        move.w      #$ 5565, HSRR1  /*mettre dans les deux Bits du CH2 du HSRR1 la
                                     valeur 2 en binaire (forcé la sortie à  l'état bas)  et
                                     mettre dans tous les autres canaux  du HSRR1 la
                                     valeur 1 (forcé les sorties a l'état haut) */
        jsr         Tempo           /*appeler le sous programme de temporisation*/

        move.w      #$ 5955, HSRR1  /*Mettre dans les deux Bits du CH5 du HSRR1 la
                                     valeur 2 en binaire (forcé la sortie à  l'état bas)  et
                                     Mettre dans tous les autres canaux  du HSRR1 la
                                     valeur 1 (forcé les sorties a l'état haut) */
        jsr         Tempo           /*appeler le sous programme de temporisation*/

        move.w      #$ 9555, HSRR1  /*Mettre dans les deux Bits du CH7 du HSRR1 la
                                     valeur 2 en binaire (forcé la sorties à  l'état bas)  et
                                     mettre dans tous les autres canaux  du HSRR1 la
                                     valeur 1 (forcé les sorties a l'état haut)  */
        jsr         Tempo           /*appeler le sous programme de temporisation*/
        bra         Prog_p          /*branchement à Prog_p  refaire le programme*/
} /* Fin du programme principal
*****************************************/
/************************
Sous Programme
*************************/
Tempo   move.l      $ 001FFFFFF,d3  /*Mettre la valeur 001FFFFFF  dans d3*/
Temp    sub.l       #1, d3          /*soustraire du registre d3 la valeur 1*/
        Bne         Temp            /* si d3 différent de 0 allée à Temp*/
        rts                         /* retour au programme principal */
```

3.1.5.4 Quelques Registres de configuration du module TPU.

Le tableau suivant résume les différents registres du module TPU, leurs noms, leurs adresses et leurs fonctions.

Access	Adresse	Nom	15 8	7 0
S	$###E00	TPUMCR	TPU module configuration register	
S	$###E02	TCR	TPU test configuration register	
S	$###E04	DSCR	Development support control register	
S	$###E06	DSSR	Development support status register	
S	$###E08	TICR	TPU Interrupt configuration register	
S	$###E0A	CIER	Channel interrupt enable register	
S	$###E0C	CFSR0	Channel function select register 0	
S	$###E0E	CFSR1	Channel function select register 1	
S	$###E10	CFSR2	Channel function select register 2	
S	$###E12	CFSR3	Channel function select register 3	
S/U	$###E14	HSQR0	Host sequence register 0	
S/U	$###E16	HSQR1	Host sequence register 1	
S/U	$###E18	HSRR0	Host service request register 0	
S/U	$###E1A	HSRR1	Host service request register 1	
S	$###E1C	CPR0	Channel priority register 0	
S	$###E1E	CPR1	Channel priority register 1	
S	$###E20	CISR	Channel interrupt status register	
S	$###E22	LR	Link register	
S	$###E24	SGLR	Service grant number register	
S	$###E26	DCNR	Decoded channel number register	
S	$###E28	TPUMCR2	TPU2 module configuration register 2 (TPU 2 only)	
S	$###F00-$###F0E		Channel 0 parameter registers	
S	$###F10-$###F1E		Channel 1 parameter register	
S	$###F20-$###F2E		Channel 2 parameter register	
S	$###F30-$###F3E		Channel 3 parameter register	
S	$###F40-$###F4E		Channel 4 parameter register	
S	$###F50-$###F5E		Channel 5 parameter register	
S	$###F60-$###F6E		Channel 6 parameter register	
S	$###F70-$###F7E		Channel 7 parameter register	
S	$###F80-$###F8E		Channel 8 parameter register	
S	$###F90-$###F9E		Channel 9 parameter register	
S	$###FA0-$###FAE		Channel 10 parameter register	
S	$###FB0-$###FBE		Channel 11 parameter register	
S	$###FC0-$###FCE		Channel 12 parameter register	
S	$###FD0-$###FDE		Channel 13 parameter register	
S	$###FE0-$###FEE		Channel 14 parameter register	
S	$###FF0-$###FFE		Channel 15 parameter register	

Tableau 3 registres du module TPU.

HSQR0 — Host Sequence Register 0 **$YFFE14**

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CH 15		CH 14		CH 13		CH 12		CH 11		CH 10		CH 9		CH 8	
RESET:															
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

HSQR1 — Host Sequence Register 1 **$YFFE16**

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CH 7		CH 6		CH 5		CH 4		CH 3		CH 2		CH 1		CH 0	
RESET:															
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

CH[15:0] — Encoded Host Sequence

The host sequence field selects the mode of operation for the time function selected on a given channel. The meaning of the host sequence bits depends on the time function specified.

Fig.49 Registres Host sequence.

Ci-dessous les registres CFSRi (i de 0 à3)

CFSR0 — Channel Function Select Register 0 **$YFFE0C**

15 … 12	11 … 8	7 … 4	3 … 0
CHANNEL15	CHANNEL14	CHANNEL13	CHANNEL12

RESET: 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

CFSR1 — Channel Function Select Register 1 **$YFFE0E**

15 … 12	11 … 8	7 … 4	3 … 0
CHANNEL11	CHANNEL10	CHANNEL9	CHANNEL8

RESET: 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

CFSR2 — Channel Function Select Register 2 **$YFFE10**

15 … 12	11 … 8	7 … 4	3 … 0
CHANNEL7	CHANNEL6	CHANNEL5	CHANNEL4

RESET: 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

CFSR3 — Channel Function Select Register 3 **$YFFE12**

15 … 12	11 … 8	7 … 4	3 … 0
CHANNEL3	CHANNEL2	CHANNEL1	CHANNEL0

RESET:

Fig 50 Registres de sélection des canaux CFSRi.

La figure suivante donne les registres de priorité CPR0 et CPR1.

CPR0 — Channel Priority Register 0 **$YFFE1C**

15 14	13 12	11 10	9 8	7 6	5 4	3 2	1 0
CH 15	CH 14	CH13	CH 12	CH 11	CH 10	CH 9	CH 8

RESET: 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

CPR1 — Channel Priority Register 1 **$YFFE1E**

15 14	13 12	11 10	9 8	7 6	5 4	3 2	1 0
CH 7	CH 6	CH 5	CH 4	CH 3	CH 2	CH 1	CH 0

RESET: 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

CH 15–0 Encoded One of Three Channel Priority Level

Bit 1	Bit 0	Service
0	0	Channel Disabled (Reset Condition)
0	1	Low
1	0	Middle
1	1	High

Fig.51 Registres de priorité CPR0 et CPR1.

HSRR0 — Host Service Request Register 0 **$###E18**

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CH 15		CH 14		CH 13		CH 12		CH 11		CH 10		CH 9		CH 8	
RESET:															
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

HSRR1 — Host Service Request Register 1 **$###E1A**

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CH 7		CH 6		CH 5		CH 4		CH 3		CH 2		CH 1		CH 0	
RESET:															
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Fig.52 Registres Host service request .

References bibliographiques.

[1] LYNN P A ˝ Introduction Digital Signal Processing. ˝ Wiley press, 1998.

[2] Lapsley.P ˝DSP processor fundamentals˝ IEEE press 1997.

[3] LILEN.H ˝ Microprocesseur du CISK au RISK ˝ Dunod paris 1995.

[4] EMMANUEL C,.Ifeachor BARRIE W.Jervis ˝ Digital signal processing a pratical approach˝ Addison-Wesley 1993.

[5] ADSP2100 Cross- Software Manual Analog Device.

[6] ADSP2101- ADSP2102 User's Manual Architecture Analog Device.

[7] Digital signal processing applications using the ADSP-2100 Family Analog device.

[8] CHASSAING. R, HORNING. D ˝ Digital signal processing with the TMS 320C25˝. A wiley –interscience publication 1990.

[9] Widrow B, John R, Glover J.R, McCool & al. Adaptive noise canceling principles and applications. Proceeding of the IEEE. 1975; 63 (12):1692-1716.

[10] Andrew Tanenbaum, Architecture de l'ordinateur, 5eme édition, Pearson Education, 2005.

[11] TPU time processor unit reference manual freescale semi conducteur INC 2004 motorola..

[12] M68300 Family MC68332 User's Manual MOTOROLA, INC. 1995.

[13] MC68332 User's Manual MOTOROLA, INC. 1990.